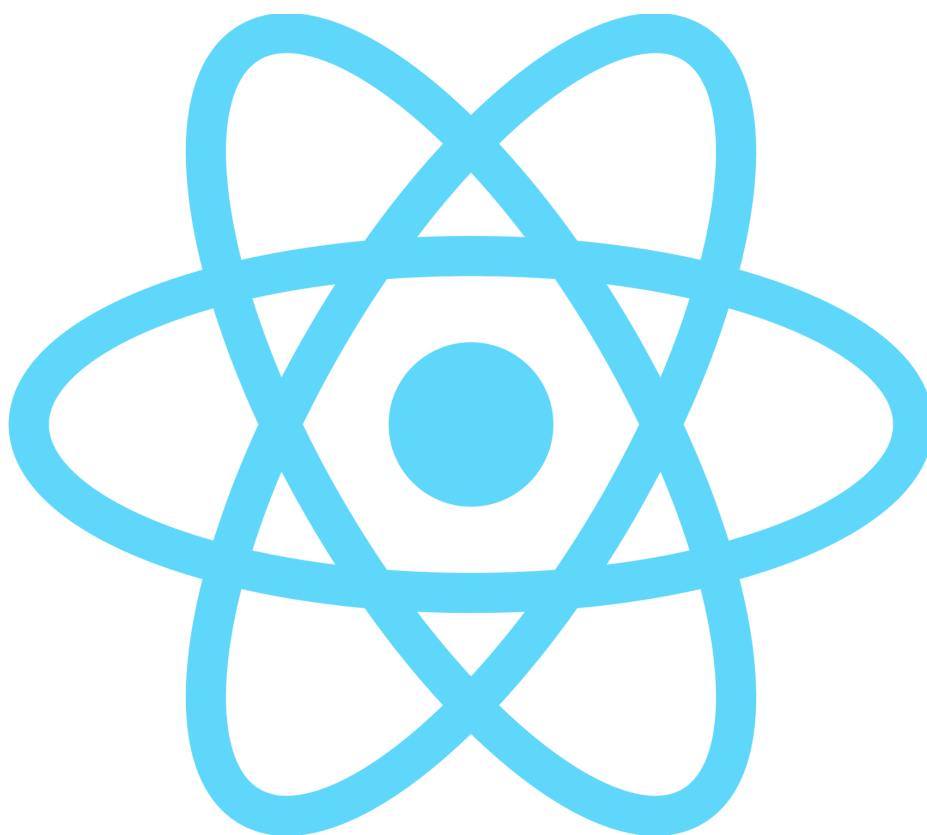


# **Učebnice React**

**k videím na YT Hackni svou budoucnost**

**David Šetek**



**Odkaz na učebnici**

**<https://bit.ly/hackni-react>**

**Seznam videí na kanále YouTube**

**David Šetek - Hackni svou budoucnost**

**[https://www.youtube.com/playlist?list=PLQ8x\\_VWW6Akua8l5spV8nHIWIG6\\_tX6dx](https://www.youtube.com/playlist?list=PLQ8x_VWW6Akua8l5spV8nHIWIG6_tX6dx)**

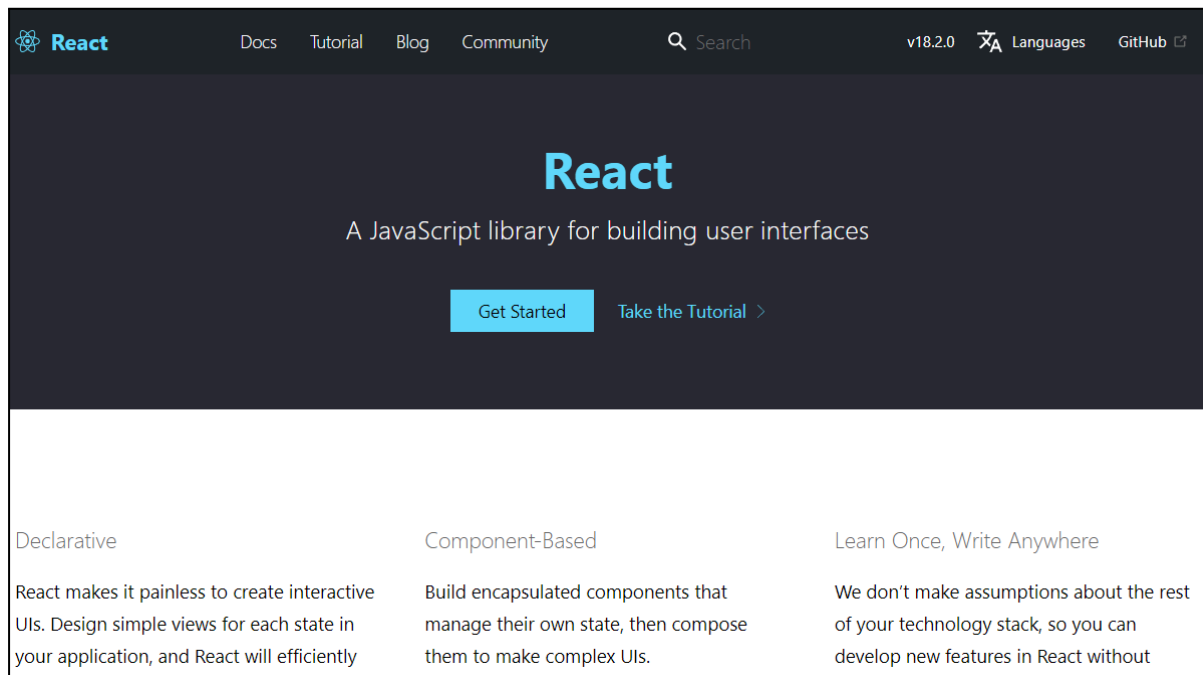
# 1. React - Co je to React

Video: <https://youtu.be/RdDzoVJOxX4>

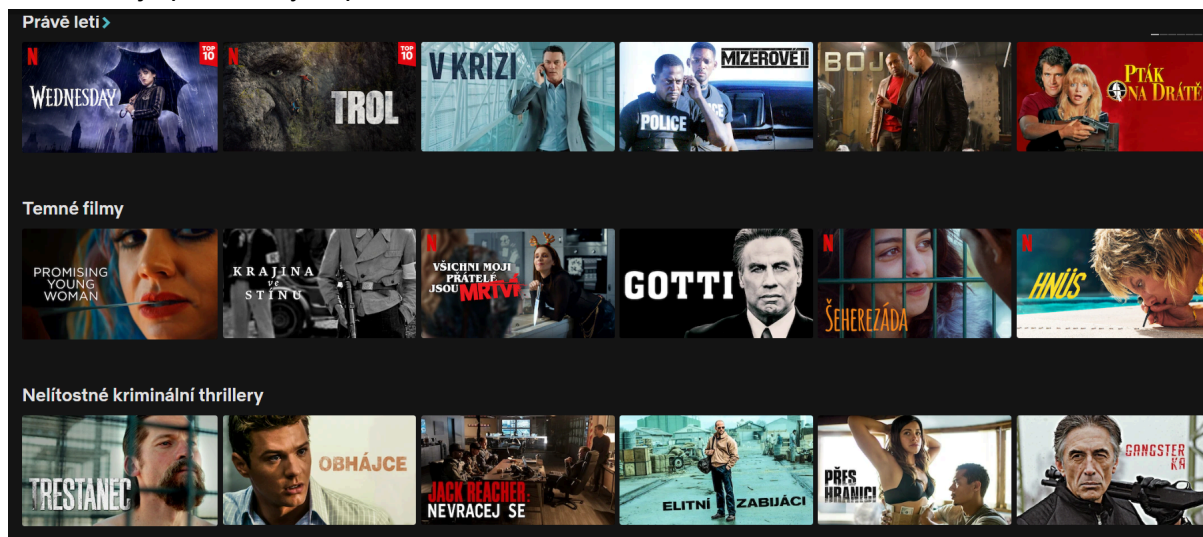
React je javascriptová knihovna pro tvorbu uživatelského prostředí.

Oficiální stránka reactu

<https://reactjs.org/>



Na reactu je postavený např. Netflix



## 2. React - Instalujeme vše potřebné pro React

Video: <https://youtu.be/d6emH27aWmq>

1. krok - prohlížeč Google: <https://www.google.com/chrome/>

2. krok - nainstalujeme VS code

Visual studio code: <https://code.visualstudio.com/>

seriál videí o VS code na kanálu Hackni svou budoucnost, pokud byste si chtěli VS code lépe nastavit

[https://www.youtube.com/watch?v=IVO49bC4YDw&list=PLQ8x\\_VWW6AksIzgE-LKqkwz4JMHh4AWVB](https://www.youtube.com/watch?v=IVO49bC4YDw&list=PLQ8x_VWW6AksIzgE-LKqkwz4JMHh4AWVB)

3. krok - nainstalujeme Node.js

<https://nodejs.org/en/>

dáme pravé tlačítko Current

4. krok - react developer tools:

<https://chrome.google.com/webstore/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi>

5. otevřeme terminál ve VS code

Terminál otevřeme: nahoře na liště **Terminal -> New Terminal**

Do terminálu napíšeme: **node -v**

Do terminálu napíšeme: **npm -v**

Obrázek z terminálu - pokud oba příkazy vyhodí nějakou verzi (číslo), tak máme oba potřebné programy nainstalované.

```
PS C:\Users\davse> node -v
v19.0.0
PS C:\Users\davse> npm -v
8.19.2
```

**Kód pro doplnění HTML (JSX) v reactu:**

```
"emmet.includeLanguages": {  
  "javascript": "javascriptreact"  
},
```

### **Příkazy pro vytvoření aplikace**

Otevřeme složku, kterou libovolně pojmenujeme

Poté dáme:

```
npx create-react-app ucime-se-react
```

```
cd ucime-se-react
```

```
npm start
```

Pokud příkaz `npx create-react-app ucime-se-react` nefunguje, tak nejdříve zkusit:

```
npx clear-npx-cache
```

### 3. React - Jaké složky a soubory nás v Reactu zajímají

Video: <https://youtu.be/gtZNGzXracs>

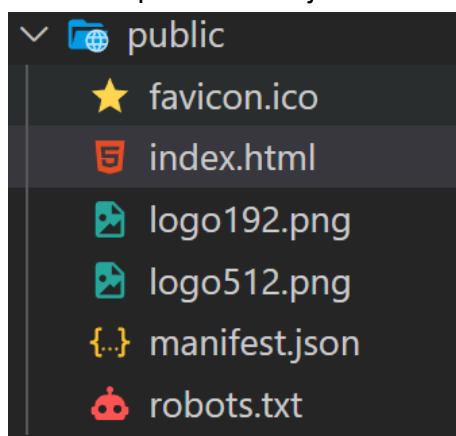
Odkaz na úvodní prezentaci

[https://docs.google.com/presentation/d/1U0Y-FkVLG-eSyyDCeJYR\\_AwDU1IAI3AD/edit?usp=sharing&oid=109973686653193365520&rtpof=true&sd=true](https://docs.google.com/presentation/d/1U0Y-FkVLG-eSyyDCeJYR_AwDU1IAI3AD/edit?usp=sharing&oid=109973686653193365520&rtpof=true&sd=true)

Důležitý slide z prezentace



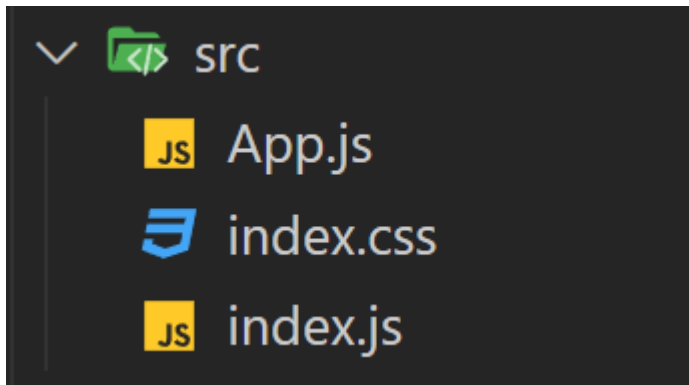
Ve složce public nás zajímá soubor index.html



Ve složce src nás zajímají soubory:

- App.js,
- index.css,
- index.js.

Zbylé soubory můžeme smazat



## 4. React - Nastavujeme důležité soubory v Reactu

Video: <https://youtu.be/jrqUEfb3R0w>

### Index.html

```
<!DOCTYPE html>
<html lang="cs">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result">

  </div>
</body>
</html>
```

### Index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

Index.css - je prázdný

App.js - je prázdný

## 5. React - Vytváříme naši první komponentu v Reactu

Video: <https://youtu.be/RAXMNs5TIA>

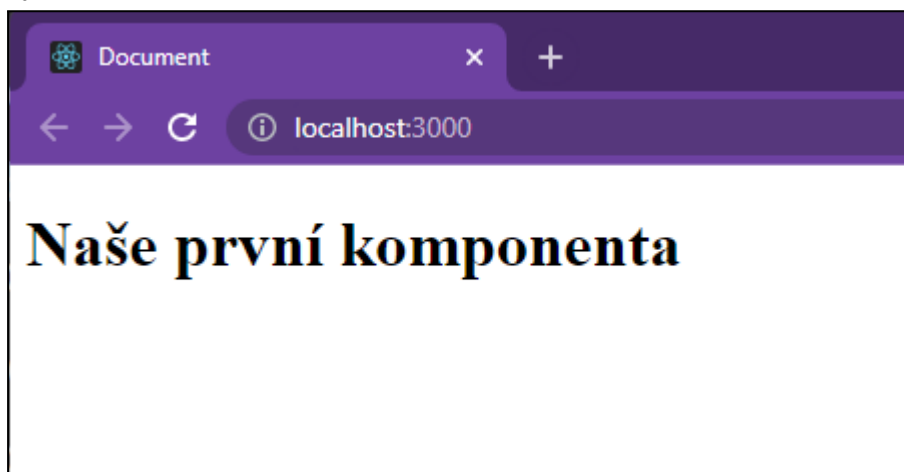
Pravidla pro tvorbu komponenty

```
// komponenta má stejný název jako je název souboru  
// komponenta obsahuje return  
// na konci souboru musí být řádek s exportem  
// v return musí něco být
```

App.js

```
const App = () => {  
  return (  
    <h1>Naše první komponenta</h1>  
  )  
}  
  
export default App
```

Výsledek na stránce



## 6. React - Return dokáže vrátit jen jeden tag

Video: <https://youtu.be/7gAT65IW6JE>

### App.js

Místo `<div></div>` můžete použít pouze `<></>`

```
const App = () => {  
  return (  
    <div>  
      <h1>Naše první komponenta</h1>  
      <p>Lorem, ipsum dolor sit amet consectetur adipisicing  
elit. Sequi maxime temporibus ab amet deserunt eaque reiciendis,  
perspiciatis at cum, perferendis quisquam neque eligendi itaque commodi  
sit officiis minus voluptatem totam?</p>  
    </div>  
  )  
}  
  
export default App  
  
// komponenta je obyčejná funkce  
// komponenta má stejný název jako je název souboru  
// komponenta obsahuje return  
// na konci souboru musí být řádek s exportem  
// v return musí něco být  
// return dokáže vrátit jen jeden tag
```

Výsledek na stránce



## 7. React - Co je to JSX?

Video: [https://youtu.be/eVq5l\\_ezygs](https://youtu.be/eVq5l_ezygs)

JSX znamená JavaScript XML. Je to to, co píšeme do return. Tedy HTML, které můžeme psát do javascriptového kódu. Správně bychom tomu HTML říkat neměli, ale měli bychom to označovat jako JSX.

App.js

Pokud do složených závorek napíšeme např. matematickou operaci, tak nám závorky vrátí výsledek.

```
const App = () => {  
  return (  
    <div>  
      <h1>Naše první komponenta</h1>  
      <h2>{2 + 2}</h2>  
      <p>Lorem, ipsum dolor sit amet consectetur adipisicing  
elit. Sequi maxime temporibus ab amet deserunt eaque reiciendis,  
perspiciatis at cum, perferendis quisquam neque eligendi itaque commodi  
sit officiis minus voluptatem totam?</p>  
    </div>  
  )  
}  
  
export default App
```

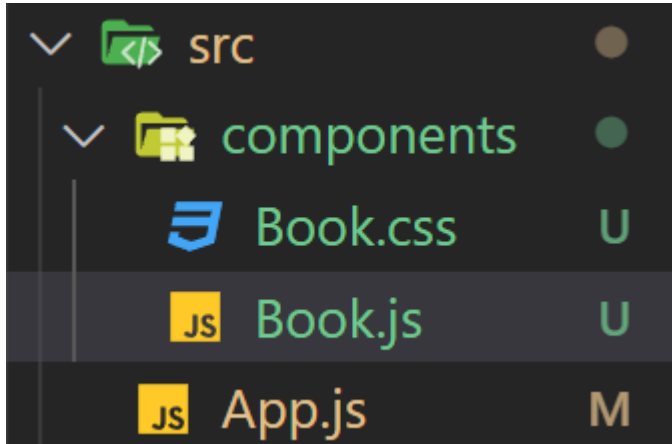
Výsledek na stránce



## 8. React - Vytváříme další komponentu

Video: <https://youtu.be/orDmXNIKtow>

Ve složce SRC založíme novou složku COMPONENTS. Do této složky vytvoříme naši první komponentu s názvem Book (kniha). Založíme dva soubory Book.js a Book.css.



Book.js

prvním krok je vždy propojit Book.js a Book.css

```
import "../Book.css"
```

Book.js

podle našich pravidel výše vytvoříme novou komponentu. Pravidla vidíte také tady dole pod kódem.

```
import "../Book.css"

const Book = () => {
  return (
    <h2>Název knihy</h2>
  )
}

export default Book

// komponenta je obyčejná funkce
// komponenta má stejný název jako je název souboru
// komponenta obsahuje return
// na konci souboru musí být řádek s exportem
// v return musí něco být
// return dokáže vrátit jen jeden tag
```

## 9. React - Stylujeme komponentu pomocí CSS a className

Video: <https://youtu.be/jdVeOiU0umA>

Book.js

K H2 jsme přidali classu. Zde používáme název className. Přřadili jsme classu s názvem main-heading.

```
import './Book.css'

const Book = () => {
  return (
    <h2 className="main-heading">Název knihy</h2>
  )
}

export default Book
```

Book.css

Provedeme běžné stylování pomocí klasických CSS pravidel.

```
.main-heading {
  color: #6c6c6c;
  font-size: 25px;
}
```

## 10. React - Zobrazujeme komponentu na stránce

Video: <https://youtu.be/ymb1QBcv5wk>

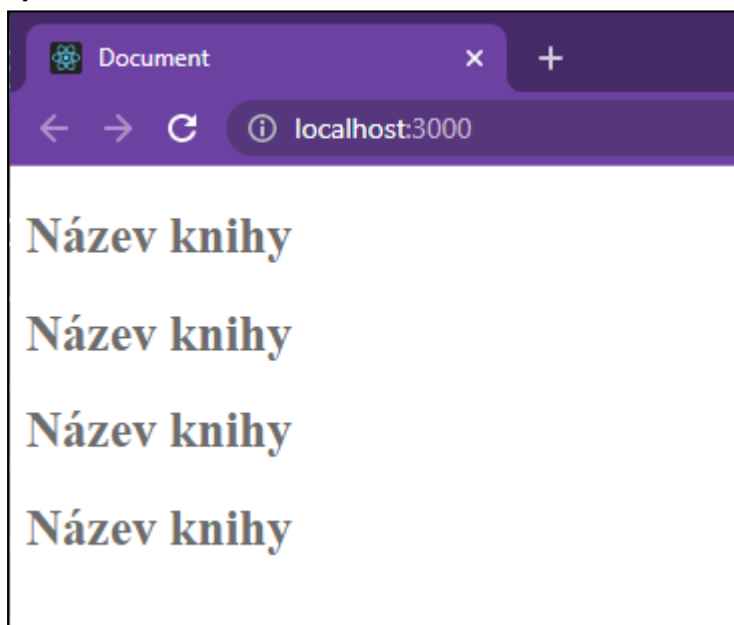
### App.js

Naimportujeme Book.js do App.js. Zvolili jsme název Book (modrý Book na prvním řádku). A nyní můžeme Book použít uvnitř komponenty App.

```
import Book from "../components/Book"
```

```
const App = () => {  
  return (  
    <div>  
      <Book />  
      <Book />  
      <Book />  
      <Book />  
    </div>  
  )  
}  
  
export default App
```

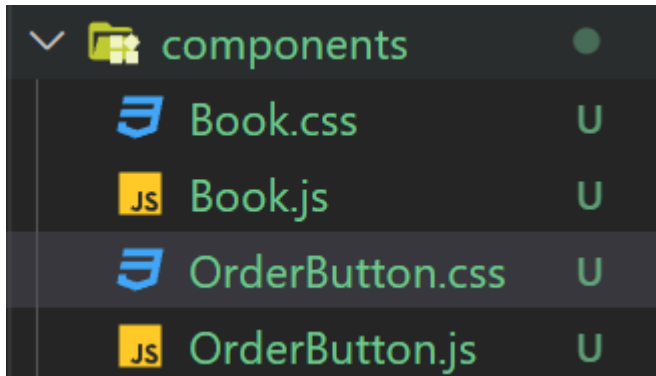
Výsledek na stránce



## 11. React - Vytváříme tlačítko jako novou komponentu

Video: <https://youtu.be/V6VvQ9NMBQg>

Ve složce COMPONENTS vytvoříme další dva nové soubory OrderButton.js a OrderButton.css



### OrderButton.js

Podle našich pravidel (viz komentáře níže) vytvoříme novou komponentu a vložíme do ní tlačítko včetně classy BTN.

```
import "../OrderButton.css"

const OrderButton = () => {
  return (
    <button type="button" className="btn">Objednat</button>
  )
}

export default OrderButton

// komponenta je obyčejná funkce
// komponenta má stejný název jako je název souboru
// komponenta obsahuje return
// na konci souboru musí být řádek s exportem
// v return musí něco být
// return dokáže vrátit jen jeden tag
```

## OrderButton.css

Classu BTN nastylujeme v OrderButton.css

```
.btn {  
  background-color: #4f4fec;  
  border-radius: 999px;  
  color: white;  
  cursor: pointer;  
  font-size: 16px;  
  border: none;  
  padding: 5px 15px;  
}
```

## 12. React - Vkládáme novou komponentu do stránky

Video: <https://youtu.be/29yADimZm4s>

### Book.js

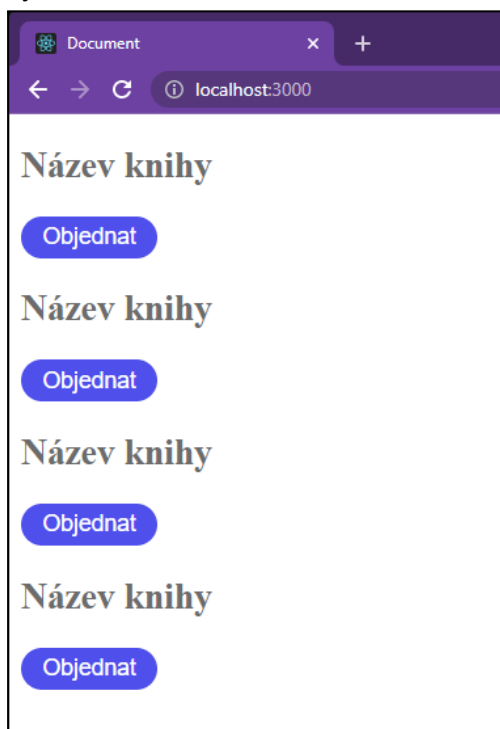
Objednávací tlačítko importujeme do Book.js, aby tlačítko bylo u každé knížky (u každého názvu)

```
import './Book.css'
import OrderButton from './OrderButton'

const Book = () => {
  return (
    <div>
      <h2 className="main-heading">Název knihy</h2>
      <OrderButton />
    </div>
  )
}

export default Book
```

Výsledek na stránce



## 13. React - Jak jednoduché jsou úpravy v komponentách

Video: <https://youtu.be/h2eSF8cBaPc>

Book.js

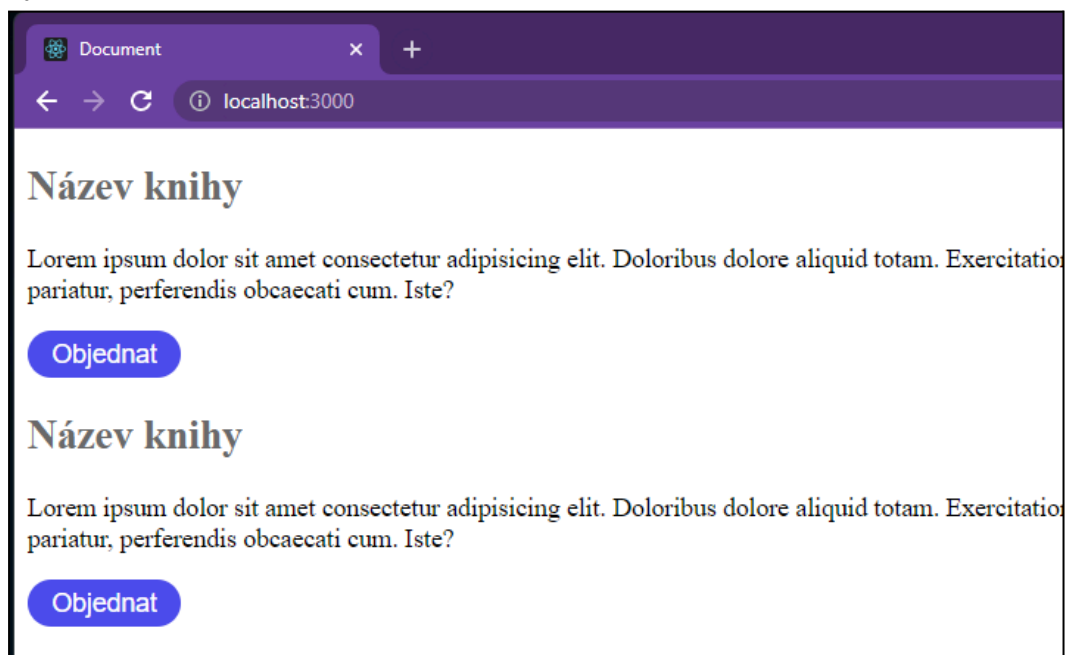
Stačí komponentu upravit (např. přidat odstavec) a vše se nám promítne do stránky ke všem komponentám Book (viz obrázek níže)

```
import './Book.css'
import OrderButton from './OrderButton'

const Book = () => {
  return (
    <div>
      <h2 className="main-heading">Název knihy</h2>
      <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.
Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et
illum aliquid aspernatur repellendus, fugit iure, reiciendis earum
pariatur, perferendis obcaecati cum. Iste?</p>
      <OrderButton />
    </div>
  )
}

export default Book
```

Výsledek na stránce



## 14. React - Stylujeme celou stránku v index.css

Video: <https://youtu.be/vOXoRc5VP6M>

### App.js

Přidáme classu pro stylování s názvem all-books

```
const App = () => {  
  return (  
    <div className="all-books">  
      <Book />  
      <Book />  
      <Book />  
      <Book />  
    </div>  
  )  
}
```

### Index.css

Stylujeme App.js

```
body {  
  background-color: rgb(199, 205, 223);  
}  
  
.all-books {  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
  flex-wrap: wrap;  
}
```

## Book.js

Přidáme classu s názvem one-book

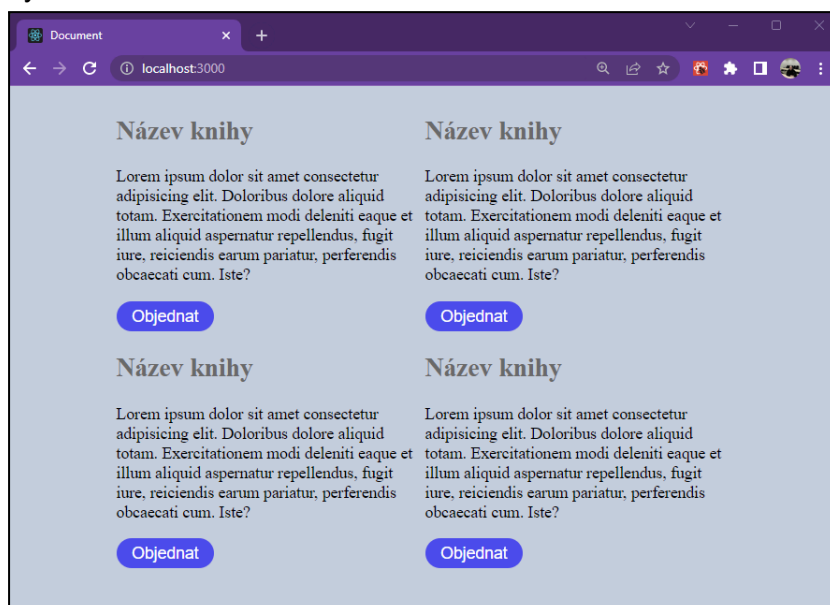
```
const Book = () => {  
  return (  
    <div className="one-book">  
      <h2 className="main-heading">Název knihy</h2>  
      <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.  
Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et  
illum aliquid aspernatur repellendus, fugit iure, reiciendis earum  
pariatur, perferendis obcaecati cum. Iste?</p>  
      <OrderButton />  
    </div>  
  )  
}
```

## Book.css

Stylujeme komponentu v Book.js

```
.one-book {  
  width: 300px;  
}  
  
.main-heading {  
  color: #6c6c6c;  
  font-size: 25px;  
}
```

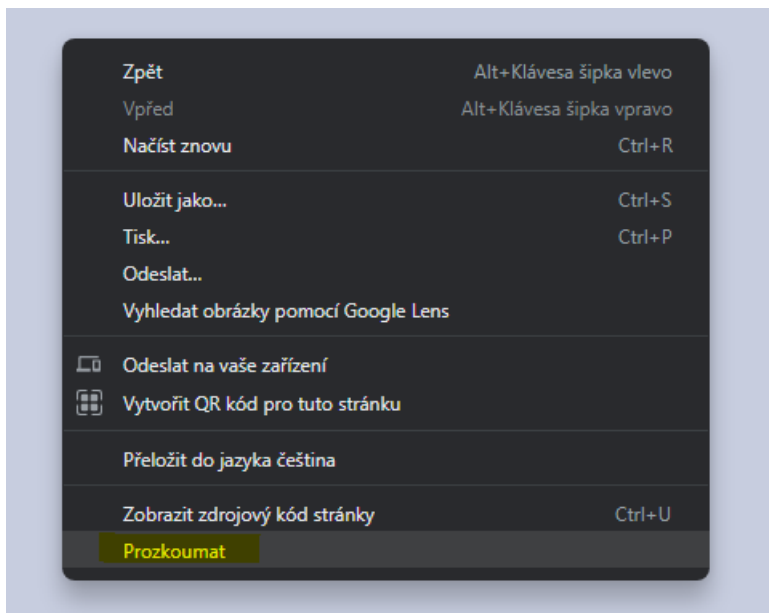
## Výsledek na stránce



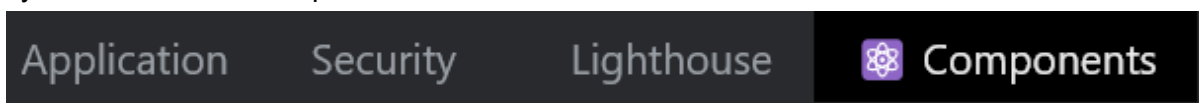
## 15. React - React developer tools - jak rychle zjistit strukturu komponent

Video: <https://youtu.be/IJhDUrWz3Kg>

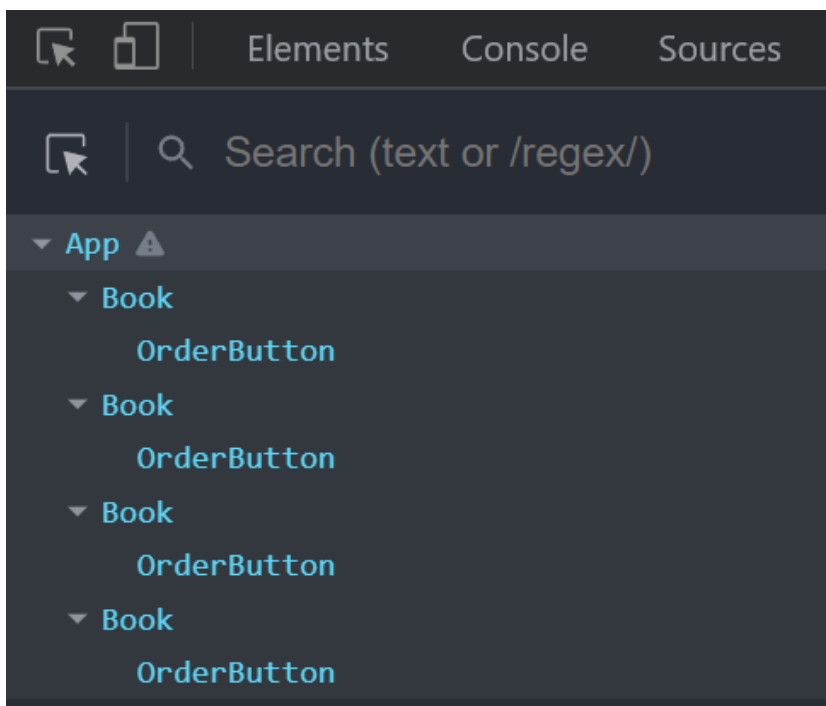
Na stránce dáme pravé tlačítko myši a zvolíme Prozkoumat (v jiných prohlížečích se tato možnost bude jmenovat jinak, ale najdete ji na stejném místě)



Vybereme záložku Components



V ní vidíme strukturu našich komponent. Do App importujeme 4x Book. Každý Book má svůj OrderButton.



## 16. React - Načítáme data, aby každá kniha byla jiná

Video: [https://youtu.be/dv8QscY\\_j8I](https://youtu.be/dv8QscY_j8I)

Data, která zkopírujeme do App.js

```
const books = [
  {
    id: 1,
    image:
"https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-kamen-mudrcu-9788000061917.jpg.webp",
    title: "Harry Potter a Kámen mudrců"
  },
  {
    id: 2,
    image:
"https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/333420609/10.jpg.webp",
    title: "Harry Potter a Tajemná komnata"
  },
  {
    id: 3,
    image:
"https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-vezen-z-azkabanu-9788000063393.jpg.webp",
    title: "Harry Potter a Vězeň z Azkabanu"
  },
]
```

## App.js

Jak to vypadá v App.js po zkopírování dat a přidání dat ke komponentám

```
const App = () => {
  const books = [
    {
      id: 1,
      image:
"https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-kamen-mudrcu-9788000061917.jpg.webp",
      title: "Harry Potter a Kámen mudrců"
    },
    {
      id: 2,
      image:
"https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/333420609/10.jpg.webp",
      title: "Harry Potter a Tajemná komnata"
    },
    {
      id: 3,
      image:
"https://www.knihydobrovsky.cz/thumbs/book-list/mod_eshop/produkty/h/harry-potter-a-vezen-z-azkabanu-9788000063393.jpg.webp",
      title: "Harry Potter a Vězeň z Azkabanu"
    },
  ]

  return (
    <div className="all-books">
      <Book myimage={books[0].image} mytitle={books[0].title} />
      <Book myimage={books[1].image} mytitle={books[1].title} />
      <Book myimage={books[2].image} mytitle={books[2].title} />
    </div>
  )
}

export default App
```

## Book.js

Do Book.js se zaslaná data promítnou pomocí props

```
const Book = (props) => {  
  return (  
    <div className="one-book">  
      <img src={props.myimage} alt="" />  
      <h2 className="main-heading">{props.mytitle}</h2>  
      <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.  
Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et  
illum aliquid aspernatur repellendus, fugit iure, reiciendis earum  
pariatur, perferendis obcaecati cum. Iste?</p>  
      <OrderButton />  
    </div>  
  )  
}  
  
export default Book
```

Výsledek na stránce

|                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                            |                                                                                                                                                            |                                                                                                                                                           |
| <b>Harry Potter a Kámen mudrců</b>                                                                                                                                                                                                           | <b>Harry Potter a Tajemná komnata</b>                                                                                                                                                                                                        | <b>Harry Potter a Vězeň z Azkabanu</b>                                                                                                                                                                                                       |
| Lorem ipsum dolor sit amet consectetur adipisicing elit. Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et illum aliquid aspernatur repellendus, fugit iure, reiciendis earum pariatur, perferendis obcaecati cum. Iste? | Lorem ipsum dolor sit amet consectetur adipisicing elit. Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et illum aliquid aspernatur repellendus, fugit iure, reiciendis earum pariatur, perferendis obcaecati cum. Iste? | Lorem ipsum dolor sit amet consectetur adipisicing elit. Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et illum aliquid aspernatur repellendus, fugit iure, reiciendis earum pariatur, perferendis obcaecati cum. Iste? |
| <a href="#">Objednat</a>                                                                                                                                                                                                                     | <a href="#">Objednat</a>                                                                                                                                                                                                                     | <a href="#">Objednat</a>                                                                                                                                                                                                                     |

## 17. React - Závěrečné video k projektu knih

Video: <https://youtu.be/2hbqWDLgqtA>

|                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |                                                                                                                                                                                                                            |
| <h3>Harry Potter a Kámen mudrců</h3> <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et illum aliquid aspernatur repellendus, fugit iure, reiciendis earum pariatur, perferendis obcaecati cum. Iste?</p> <p>Objednat</p> | <h3>Harry Potter a Tajemná komnata</h3> <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et illum aliquid aspernatur repellendus, fugit iure, reiciendis earum pariatur, perferendis obcaecati cum. Iste?</p> <p>Objednat</p> | <h3>Harry Potter a Vězeň z Azkabanu</h3> <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Doloribus dolore aliquid totam. Exercitationem modi deleniti eaque et illum aliquid aspernatur repellendus, fugit iure, reiciendis earum pariatur, perferendis obcaecati cum. Iste?</p> <p>Objednat</p> |

## 18. React - Vytváříme novou složku a zprovozňujeme React

Video: <https://youtu.be/qJqoJCnlueE>

Příkaz na vytvoření nové složky pro zprovoznění Reactu

`npx create-react-app moje-app`

### Index.html

```
<!DOCTYPE html>
<html lang="cs">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result">

  </div>
</body>
</html>
```

### Index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

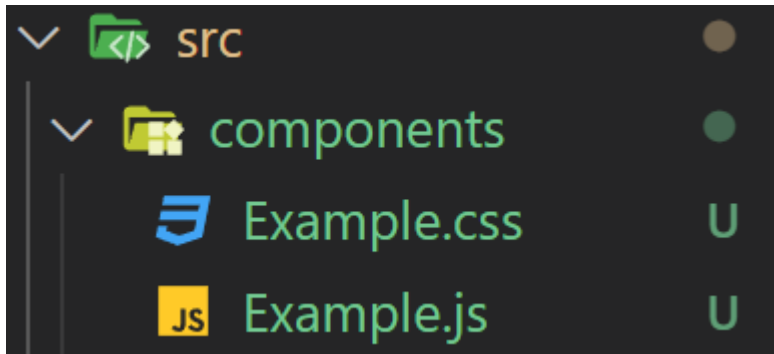
Index.css - je prázdný

App.js - je prázdný

## 19. React - Opakujeme vytváření komponenty

Video: <https://youtu.be/sDCYOff1PUs>

Ve složce SRC založíme složku COMPONENTS. A do ní vytvoříme dva soubory - Example.js a Example.css



### Example.js

Vytvoříme běžným způsobem komponentu, která má zatím v sobě jen H1 s libovolným textem

```
import "../Example.css"

const Example = () => {
  return (
    <h1>Nic moc</h1>
  )
}

export default Example

// komponenta je obyčejná funkce
// komponenta má stejný název jako je název souboru
// komponenta obsahuje return
// na konci souboru musí být řádek s exportem
// v return musí něco být
// return dokáže vrátit jen jeden tag
```

**Example.css** - je prázdné

## App.js

Naimportujeme Example a použijeme ho jednou v return

```
import Example from "../components/Example"

const App = () => {
  return (
    <Example />
  )
}

export default App

// komponenta je obyčejná funkce
// komponenta má stejný název jako je název souboru
// komponenta obsahuje return
// na konci souboru musí být řádek s exportem
// v return musí něco být
// return dokáže vrátit jen jeden tag
```

## 20. React - Událost kliknutí na tlačítko

Video: <https://youtu.be/3jpEofDQCXg>

Example.js

Přidáme proměnnou title a její text chceme zobrazovat v H2. Poté si přidáme tlačítko. Aby tlačítko něco dělalo, tak na něj přidáme událost kliknutí (onClick) a funkci, kterou má kliknutí spustit. Tuto funkci s názvem buttonHandler jsme nadefinovali nahoře. Má vypsát do konzole text "Kliknuto".

```
const Example = () => {  
  let title = "Můj první title"  
  
  const buttonHandler = () => {  
    console.log("Kliknuto")  
  }  
  
  return (  
    <div>  
      <h2>{title}</h2>  
      <button type="button" onClick={buttonHandler}>Změnit  
title</button>  
    </div>  
  )  
}
```

Výsledek na stránce

**Můj první title**

Změnit title

## 21. React - Co je to destructuring

Video: <https://youtu.be/WETp0ifNih0>

Zde vidíme příklad destructuringu. Prvky z vychoziPole přidělíme jednotlivým proměnným student1, student2 a student3. Ve student1 je tedy David, ve student2 je Hermiona a ve student3 je Harry.

```
// čistý javascript - začátek
const vychoziPole = ["David", "Hermiona", "Harry"]
// const student1 = vychoziPole[0]
// const student2 = vychoziPole[1]
// const student3 = vychoziPole[2]

const [student1, student2, student3] = vychoziPole
// čistý javascript - konec
```

## 22. React - useState a jak změnit text nadpisu

Video: <https://youtu.be/fWI2LdW8R9o>

Nahoře naimportujeme useState.

Dále useState použijeme pro nastavení proměnné title na hodnotu "Muj první title".

Funkce setTitle dělá to, že do ní pošleme nějakou hodnotu a ona ji nastaví do proměnné title. Ale k tomu ještě zabezpečí refreshnutí komponenty. Takže pokud se title používá v return, tak zde dojde ke znovunačtení a title se přepíše.

```
import './Example.css'
import { useState } from 'react'

const Example = () => {
  console.log("TEST")
  const [title, setTitle] = useState("Muj první title")

  const buttonHandler = () => {
    setTitle("Nový text v titlu")
  }

  return (
    <div>
      <h2>{title}</h2>
      <button type="button" onClick={buttonHandler}>Změnit
title</button>
    </div>
  )
}
```

## 23. React - Procvičujeme useState

Video: <https://youtu.be/lxkr3HrJVJI>

Example.js

Přidáme další tlačítko. Když se na něj klikne, tak se spustí funkce BUTTON2HANDLER. Tam nastaví pomocí SETTITLE proměnnou TITLE na "xxxxxxx".

```
const Example = () => {
  const [title, setTitle] = useState("Muj první title")

  const buttonHandler = () => {
    setTitle("Nový text v titlu")
  }

  const button2Handler = () => {
    setTitle("xxxxxxxxxxxxx")
  }

  return (
    <div>
      <h2>{title}</h2>
      <button type="button" onClick={buttonHandler}>Změnit
title</button>
      <button type="button" onClick={button2Handler}>Opět
změnit title</button>
    </div>
  )
}
```

Výsledek na stránce

Po kliknutí na Změnit title se text změní na "Nový text v titlu" a po kliknutí na druhé tlačítko Opět změnit title, tak se text změní na "xxxxxxxxxxxxx".



## 24. React - Znovu procvičujeme useState a mažeme title

Video: <https://youtu.be/-nmPoMkGw7E>

Přidáme si ještě jedno tlačítko, které zavolá funkci deleteAll a ta vymaže obsah titlu. Respektive ho nastaví na prázdný string.

```
const deleteAll = () => {  
    setTitle("")  
}  
  
return (  
    <div>  
        <h2>{title}</h2>  
        <button type="button" onClick={buttonHandler}>Změnit  
title</button>  
        <button type="button" onClick={button2Handler}>Opět změnit  
title</button>  
        <button type="button" onClick={deleteAll}>Vymazat  
title</button>  
    </div>  
)
```

Po kliknutí na Vymazat title se obsah H2 vymaže, protože title se nastaví na prázdný string.

**Muj první title**

Změnit title

Opět změnit title

Vymazat title

## 25. React - A znovu procvičujeme useState

Video: <https://youtu.be/i4AIRB7gXCs>

Pokud se klikne na poslední tlačítko, tak se zavolá funkce buttonChange, která mění obsah proměnné buttonText na "Nový text tlačítka".

```
const Example = () => {  
  const [title, setTitle] = useState("Muj první title")  
  const [buttonText, setButtonText] = useState("Původní text  
tlačítka")  
  
  const buttonHandler = () => {  
    setTitle("Nový text v titlu")  
  }  
  
  const button2Handler = () => {  
    setTitle("xxxxxxxxxxxxx")  
  }  
  
  const deleteAll = () => {  
    setTitle("")  
  }  
  
  const buttonChange = () => {  
    setButtonText("Nový text tlačítka")  
  }  
  
  return (  
    <div>  
      <h2>{title}</h2>  
      <button type="button" onClick={buttonHandler}>Změnit  
title</button>  
      <button type="button" onClick={button2Handler}>Opět změnit  
title</button>  
      <button type="button" onClick={deleteAll}>Vymazat  
title</button>  
      <button type="button"  
onClick={buttonChange}>{buttonText}</button>  
    </div>  
  )  
}
```

Výsledek na stránce před kliknutím na poslední tlačítko

# Muj první title

Změnit title

Opět změnit title

Vymazat title

Původní text tlačítka

Po kliknutí na poslední tlačítko

# Muj první title

Změnit title

Opět změnit title

Vymazat title

Nový text tlačítka

## 26. React - Stylujeme aplikaci pomocí CSS

Video: [https://youtu.be/F\\_SnAtday2A](https://youtu.be/F_SnAtday2A)

Odkaz na barevné palety: <https://color.adobe.com/cs/explore>

### App.js

Přidáme classu changer a obalující DIV

```
const App = () => {  
  return (  
    <div className="changer">  
      <Example />  
    </div>  
  )  
}
```

### Index.css

```
body {  
  background-color: #011526;  
}  
  
.changer {  
  height: 100vh;  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
  align-items: center;  
}
```

### Example.js

Přidáme classu text-button-changer

```
return (  
  <div className="text-button-changer">  
    <h2>{title}</h2>  
    <button type="button" onClick={buttonHandler}>Změnit  
title</button>  
    <button type="button" onClick={button2Handler}>Opět změnit  
title</button>  
    <button type="button" onClick={deleteAll}>Vymazat  
title</button>  
    <button type="button"  
onClick={buttonChange}>{buttonText}</button>  
  </div>  
)
```

### Example.css

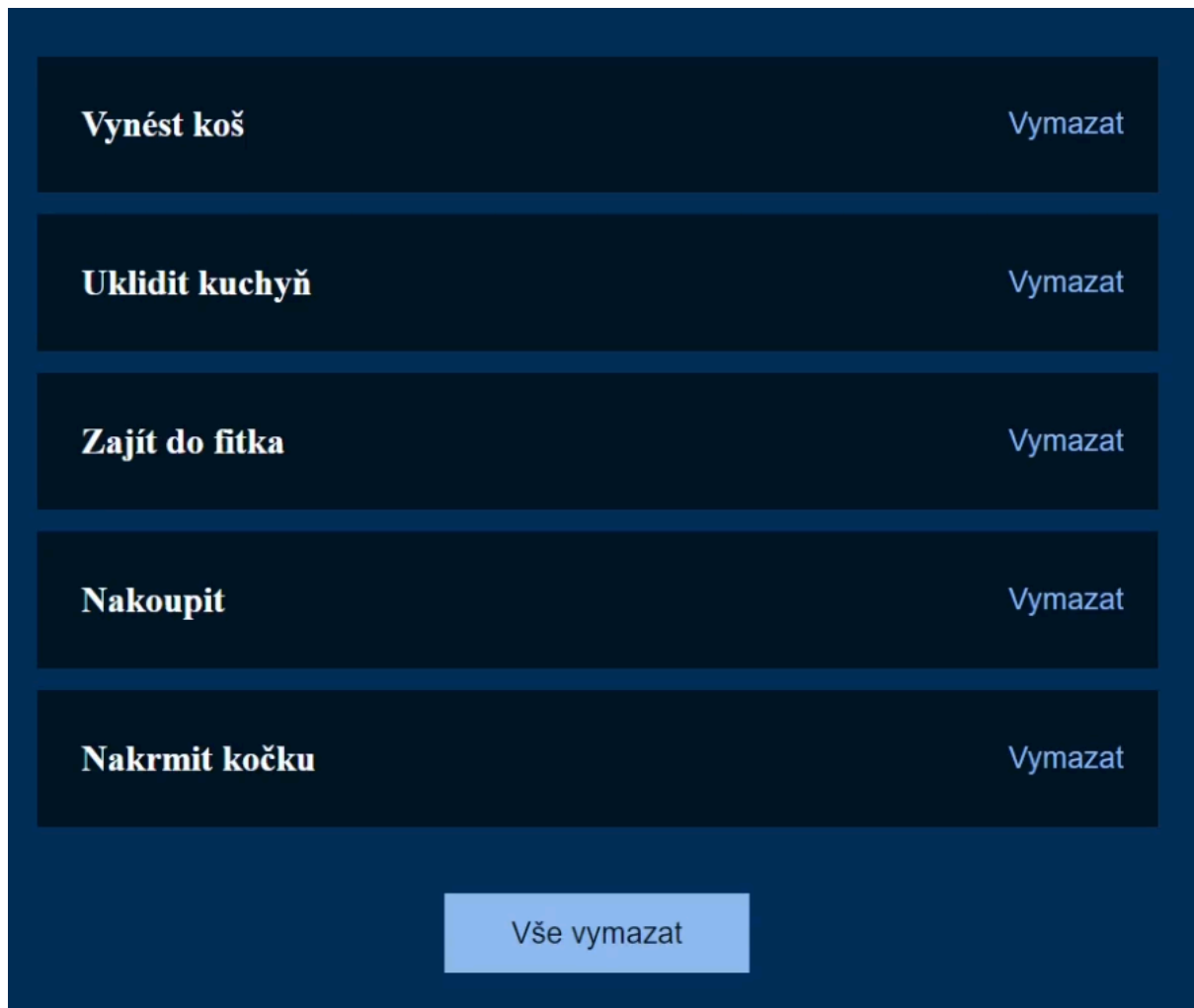
```
.text-button-changer {  
  display: flex;  
  flex-direction: column;  
}  
  
.text-button-changer h2 {  
  text-align: center;  
  color: #91BBF2;  
}  
  
.text-button-changer button {  
  background-color: #035AA6;  
  color: #91BBF2;  
  border: none;  
  margin: 5px 0px;  
  padding: 3px 0px;  
}
```

Výsledek na stránce



## 27. React - Ukázka aplikace

Video: <https://youtu.be/Z0EKvtpP3l8>



## 28. React - Vytváříme složku pro nový projekt

Video: <https://youtu.be/yZHFqRXmInE>

### Index.html

```
<!DOCTYPE html>
<html lang="cs">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result">

  </div>
</body>
</html>
```

### Index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

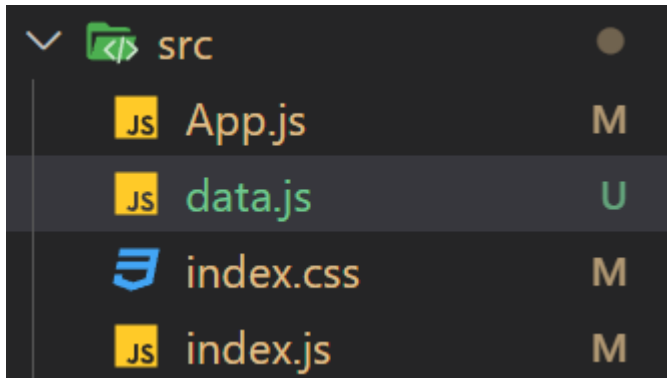
Index.css - je prázdný

App.js - je prázdný

## 29. React - Vkládáme data a nastavuje App

Video: <https://youtu.be/DQrd1DbLRrQ>

Ve složce SRC si založíme nový soubor DATA.JS



data.js

Do DATA.JS umístíme tato data, která představují seznam úkolů

```
const data = [  
  { id: 1, name: 'Vynést koš' },  
  { id: 2, name: 'Uklidit kuchyň' },  
  { id: 3, name: 'Zajít do fitka' },  
  { id: 4, name: 'Nakoupit' },  
  { id: 5, name: 'Nakrmit kočku' },  
]  
  
export default data
```

App.js

Založíme běžnou komponentu (funkci)

```
const App = () => {  
  // čistý javascript - začátek  
  
  // čistý javascript - konec  
  return (  
    <h1>Test</h1>  
  )  
}  
  
export default App
```

## 30. React - Funkce map a jak pracovat s daty

Video: <https://youtu.be/pCoDMQcp6Vo>

Funkce map vezme jedno pole a vytvoří z něj výsledné pole, která je nějak upravené. Vezme každý prvek původního pole, provede úpravy a prvky pak dá do nového pole.

App.js

```
// čistý javascript - začátek

// 1. příklad
const vychoziPole = [5, 6, 7, 8, 9]

const vyslednePole = vychoziPole.map( (jednoCislo) => {
  return jednoCislo + 10
})

console.log(vyslednePole)

// 2. příklad
const data = [
  { id: 1, name: 'Vynést koš' },
  { id: 2, name: 'Uklidit kuchyň' },
  { id: 3, name: 'Zajít do fitka' },
  { id: 4, name: 'Nakoupit' },
  { id: 5, name: 'Nakrmit kočku' },
]

const vysledek = data.map( (jedenUkol) => {
  return jedenUkol.name
})

console.log(vysledek)

// čistý javascript - konec
```

Výsledek v konzoli

```
► (5) [15, 16, 17, 18, 19]
► (5) ['Vynést koš', 'Uklidit kuchyň', 'Zajít do fitka', 'Nakoupit', 'Nakrmit kočku']
```

## 31. React - Funkce filter a kdy ji budeme používat

Video: <https://youtu.be/LCuAH2wKXBc>

Funkce filter vezme první prvek z pole (nultý prvek, pokud to budeme podle indexů) a

App.js

```
// čistý javascript - začátek
const data = [
  { id: 1, name: 'Vynést koš' },
  { id: 2, name: 'Uklidit kuchyň' },
  { id: 3, name: 'Zajít do fitka' },
  { id: 4, name: 'Nakoupit' },
  { id: 5, name: 'Nakrmit kočku' },
]

const vymazeneId = 4

const vyslednePole = data.filter( (jedenUkol) => {
  return jedenUkol.id !== vymazeneId
})

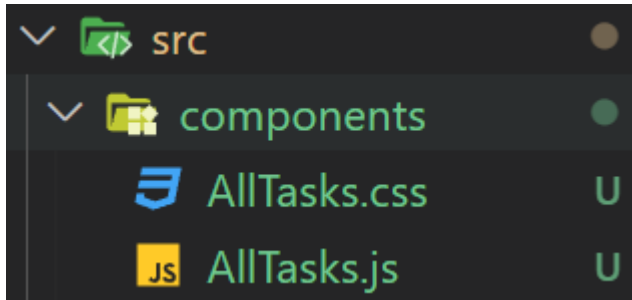
console.log(vyslednePole)

// čistý javascript - konec
```

## 32. React - Vytváříme novou komponentu a vypisujeme data do stránky pomocí map

Video: <https://youtu.be/1QfCUs6XgFA>

Ve složce SRC vytvoříme složku COMPONENTS. V ní vytvoříme dva soubory. AllTasks.js a AllTasks.css



### AllTasks.js

Naimportujeme CSS, ale také data ze souboru data.js

V komponentě AllTasks použijeme funkci MAP k tomu, abychom postupně vypsalí jednotlivé úkoly z DATA do stránky.

```
import "../AllTasks.css"
import data from "../data"

const AllTasks = () => {
  return <div>
    {
      data.map( (oneTask) => {
        const {id, name} = oneTask

        return <div className="one-task">
          <h4>{name}</h4>
        </div>
      })
    }
  </div>
}

export default AllTasks
```

## App.js

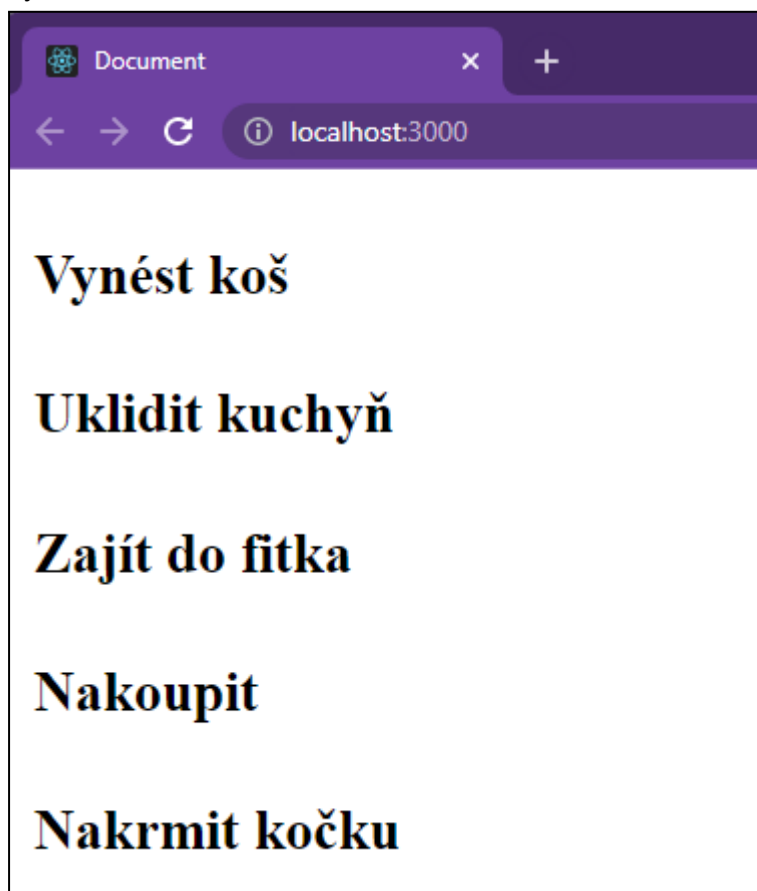
Zde naimportujeme komponentu ze souboru AllTasks.js a přidáme ji do App.

```
import AllTasks from "../components/AllTasks"
```

```
const App = () => {  
  return (  
    <AllTasks/>  
  )  
}
```

```
export default App
```

Výsledek na stránce



### 33. React - Unikátní identifikátor, přidání tlačítka a zachycení kliknutí

Video: <https://youtu.be/p1hFZdhlwVE>

Jako první přidáme k výpisu každého úkolu unikátní identifikátor (console v prohlížeči si stěžuje, že každá položka seznamu musí mít identifikátor KEY, tak k tomu využijeme ID). Poté přidáme tlačítko a když se na něj klikne, tak to spustí funkci taskHandler, kterou máme nadefinovanou na začátku komponentu a do konzole se vypíše slovo "klik".

```
const AllTasks = () => {
  const taskHandler = () => {
    console.log("klik")
  }

  return <div>
    {
      data.map( (oneTask) => {
        const {id, name} = oneTask

        return <div className="one-task" key={id}>
          <h4>{name}</h4>
          <button type="button"
onClick={taskHandler}>Vymazat</button>
        </div>
      })
    }
  </div>
}

export default AllTasks
```

Výsledek na stránce

**Vynést koš**

**Uklidit kuchyň**

**Zajít do fitka**

## 34. React - Mažeme úkol, na který bylo kliknuto

Video: [https://youtu.be/X0J\\_nOg2yYI](https://youtu.be/X0J_nOg2yYI)

Naimportujeme useState úplně nahoře

Pomocí useState nastavíme do proměnné myTask proměnnou DATA.

Jakmile někdo klikne na tlačítko, tak se doplní do ID jeho číslo a to zavolá funkci TASKHANDLER. Ta profiltruje všechny položky z data a vyndá do FILTEREDTASKS pouze ty, které nemají ID, které se rovná ID, na které bylo kliknuto.

SETMYTASK nastaví vyfiltrované pole do proměnné MYTASK. useState zabezpečí refreshnutí, takže do stránky se vypíší pouze zbylé prvky, které jsou uloženy v MYTASK.

```
import './AllTasks.css'
import data from '../data'
import { useState } from 'react'

const AllTasks = () => {
  const [myTask, setMyTask] = useState(data)

  const tasksHandler = (idecko) => {
    const filteredTasks = myTask.filter( (oneTask) => {
      return oneTask.id !== idecko
    })

    setMyTask(filteredTasks)
  }

  return <div>
    {
      myTask.map( (oneTask) => {
        const {id, name} = oneTask

        return <div className="one-task" key={id}>
          <h4>{name}</h4>
          <button type="button" onClick={() =>
tasksHandler(id)}>Vymazat</button>
          </div>
        })
      )
    }
  </div>
}

export default AllTasks
```

## 35. React - Tlačítko na vymazání všech úkolů

Video: <https://youtu.be/vEWrydlk9AU>

### AllTasks.js

Přidáme další tlačítko (až za funkci MAP). Po kliknutí na tlačítko se spustí funkce ALLDELETEHANDLER, která nastaví MYTASK na hodnotu prázdného pole (pomocí funkce SETMYTASK). A useState se postará o refreshnutí komponenty. Takže se do stránky nevypíše nic - vše je vymazáno.

```
const AllTasks = () => {
  const [myTask, setMyTask] = useState(data)

  const tasksHandler = (idecko) => {
    const filteredTasks = myTask.filter( (oneTask) => {
      return oneTask.id !== idecko
    })

    setMyTask(filteredTasks)
  }

  const allDeleteHandler = () => {
    setMyTask([])
  }

  return <div>
    {
      myTask.map( (oneTask) => {
        const {id, name} = oneTask

        return <div className="one-task" key={id}>
          <h4>{name}</h4>
          <button type="button" onClick={() =>
tasksHandler(id)}>Vymazat</button>
        </div>
      })
    }

    <button type="button" className="main-button"
onClick={allDeleteHandler}>Vše vymazat</button>
  </div>
}
```

Dole na stránce vidíme tlačítko Vše vymazat. Po kliknutí se vymažou ze stránky všechny úkoly.

## **Nakoupit**

Vymazat

## **Nakrmit kočku**

Vymazat

Vše vymazat

## 36. React - Stylujeme aplikaci na úkoly

Video: <https://youtu.be/lhklhNzUVCQ>

Barevné palety: <https://color.adobe.com/cs/explore>

### App.js

Do App.js přidáme classu s názvem ALL-TASKS

```
import AllTasks from "../components/AllTasks"

const App = () => {
  return (
    <div className="all-tasks">
      <AllTasks />
    </div>
  )
}

export default App
```

### index.css - základní stylování

```
body {
  background-color: #023059;
  color: white;
}

.all-tasks {
  margin-top: 50px;
}
```

### AllTasks.css

```
.one-task {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  background-color: #011526;
  width: 500px;
  margin: 10px auto;
}

.one-task h4 {
  padding-left: 20px;
}
```

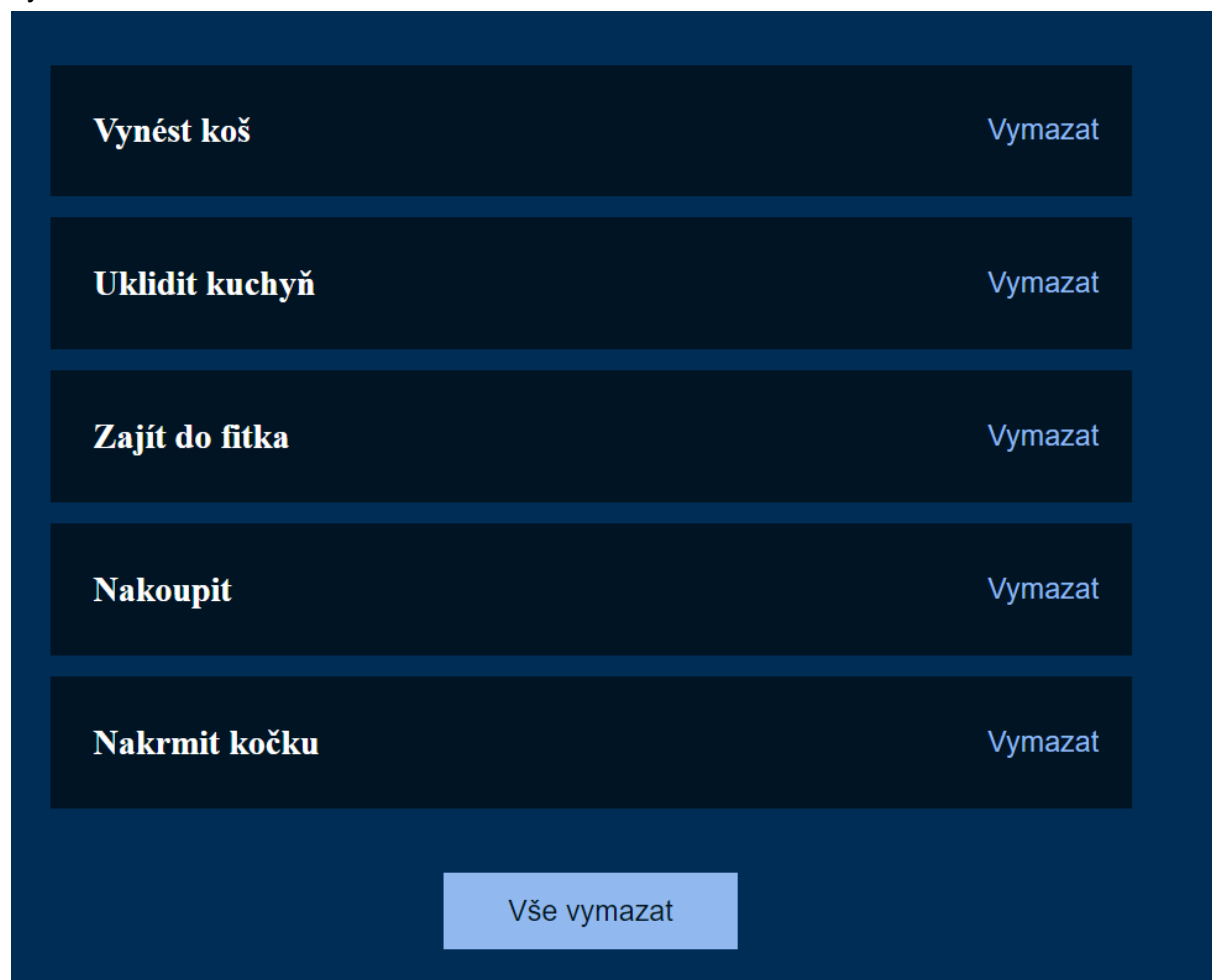
```

.one-task button {
  background-color: #011526;
  color: #91BBF2;
  border: none;
  padding-right: 15px;
  cursor: pointer;
}

.main-button {
  background-color: #91BBF2;
  color: #011526;
  border: none;
  padding: 10px 30px;
  cursor: pointer;
  display: block;
  margin: 30px auto;
}

```

Výsledek na stránce



## 37. React - Počítadlo a nastavení aplikace

Video: <https://youtu.be/T5PpQvAhRY0>

### Index.html

Vložíme DIV s ID result, do kterého budeme obsah aplikace vkládat

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <div id="result"></div>
</body>
</html>
```

### Index.js

Zkopírujeme tento základní kód

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

### App.js

Vytvoříme základní komponentu.

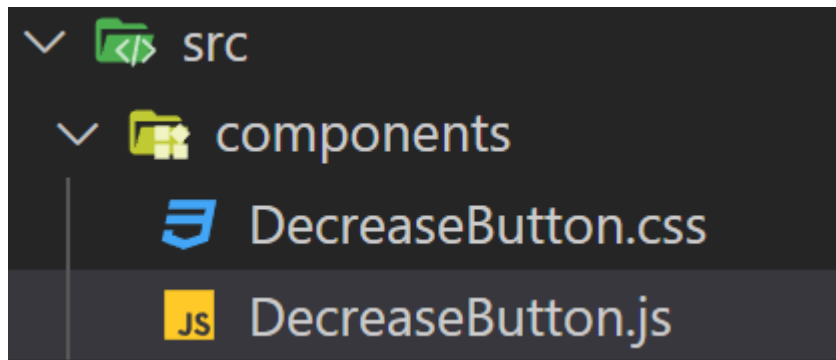
```
const App = () => {
  return (
    <h1>Test</h1>
  )
}

export default App
```

## 38. React - Vytváříme odečítací tlačítko

Video: [https://youtu.be/bDawZ\\_67iZY](https://youtu.be/bDawZ_67iZY)

Ve složce SRC si založíme složku COMPONENTS a v ní dva soubory DecreaseButton.js a DecreaseButton.css



### DecreaseButton.js

Vytvoříme běžnou komponentu s tlačítkem, které zatím nic nedělá

```
import "../DecreaseButton.css"

const DecreaseButton = () => {
  return (
    <button>Snížit o 1</button>
  )
}

export default DecreaseButton
```

### App.js

Do App.js naimportujeme DecreaseButton a vyplníme ho do return. Navíc vytvoříme proměnnou VALUE s hodnotou 0 a promítneme do stránky do H2. Ještě přidáme úvodní nadpis do H1.

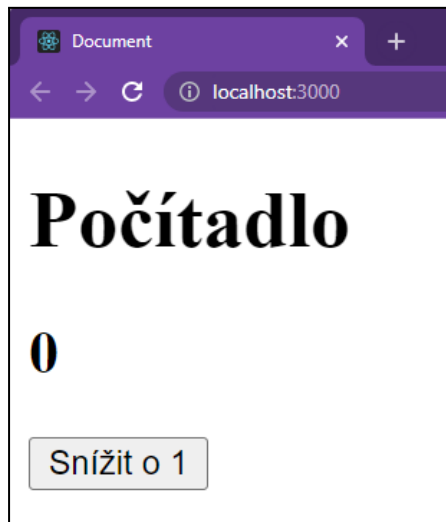
```
import DecreaseButton from "../components/DecreaseButton"

const App = () => {
  const value = 0

  return (
    <div>
      <h1>Počítadlo</h1>
      <h2>{value}</h2>
      <DecreaseButton />
    </div>
  )
}
```

```
}  
  
export default App
```

Výsledek na stránce



## 39. React - Připravujeme useState

Video: <https://youtu.be/DFiwNEPpUAW>

Odkaz použitý ve videu:

<https://reactjs.org/docs/hooks-reference.html#useState>

### App.js

```
import DecreaseButton from "../components/DecreaseButton"
import { useState } from "react"

const App = () => {
  const [value, setValue] = useState(0)

  return (
    <div>
      <h1>Počítadlo</h1>
      <h2>{value}</h2>
      <DecreaseButton />
    </div>
  )
}

export default App
```

## 40. React - Propojení useState, props a kliknutí na tlačítko

Video: <https://youtu.be/rV9nDZpr7pQ>

### App.js

Vytvoříme funkci, která bude pomocí SETVALUE nastavovat VALUE na novou hodnotu - vždy sníženou o 1. SETVALUE také zabezpečí opětovné vypsání stránky, takže v H2 se objeví nová hodnota VALUE.

Funkci DECREASEONE musíme poslat do další komponenty pomocí props.

```
import DecreaseButton from "../components/DecreaseButton"
import { useState } from "react"

const App = () => {
  const [value, setValue] = useState(0)

  const decreaseOne = () => {
    const newValue = value - 1
    setValue(newValue)
  }

  return (
    <div>
      <h1>Počítadlo</h1>
      <h2>{value}</h2>
      <DecreaseButton decrease={decreaseOne}/>
    </div>
  )
}

export default App
```

### DecreaseButton.js

Zde pomocí props zavoláme DECREASE, pod kterým se skrývá funkce DECREASEONE z App.js

```
import "../DecreaseButton.css"

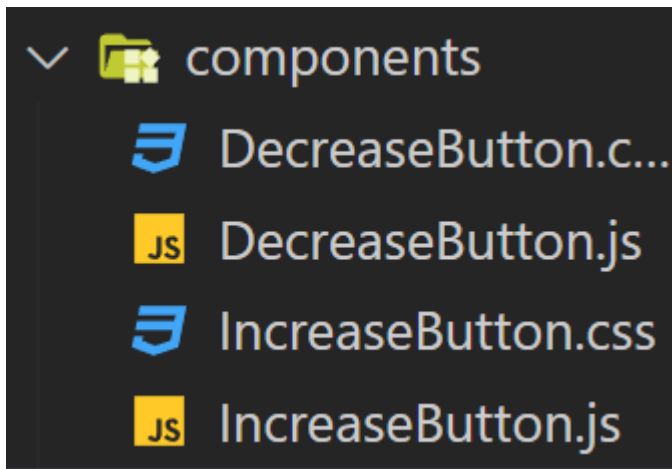
const DecreaseButton = (props) => {
  return (
    <button onClick={props.decrease}>Snížit o 1</button>
  )
}

export default DecreaseButton
```

## 41. React - Vytváříme a nastavujeme tlačítko pro zvýšení

Video: <https://youtu.be/SXMkaHp9GZc>

Ve složce COMPONENTS založíme další dva soubory IncreaseButton.js a IncreaseButton.js



### App.js

Naimportujeme INCREASEBUTTON a použijeme v return. Vytvoříme funkci, která bude zvyšovat VALUE o 1. Pojmenovali jsme ji INCREASEONE. Pošleme ji do INCREASEBUTTON pod názvem INCREASE.

```
import DecreaseButton from "../components/DecreaseButton"
import IncreaseButton from "../components/IncreaseButton"
import { useState } from "react"

const App = () => {
  const [value, setValue] = useState(0)

  const decreaseOne = () => {
    const newValue = value - 1
    setValue(newValue)
  }

  const increaseOne = () => {
    const newValue = value + 1
    setValue(newValue)
  }

  return (
    <div>
      <h1>Počítadlo</h1>
      <h2>{value}</h2>
      <DecreaseButton decrease={decreaseOne}/>
      <IncreaseButton increase={increaseOne}/>
    </div>
  )
}
```

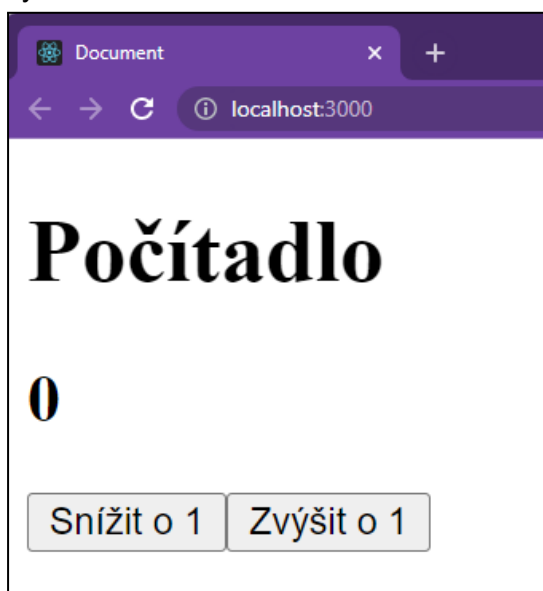
```
    </div>  
  )  
}
```

### IncreaseButton.js

Zde si INCREASE přebereme pomocí props od App.js a použijeme při kliknutí na tlačítko.

```
import './IncreaseButton.css'  
  
const IncreaseButton = (props) => {  
  return (  
    <button onClick={props.increase}>Zvýšit o 1</button>  
  )  
}  
  
export default IncreaseButton
```

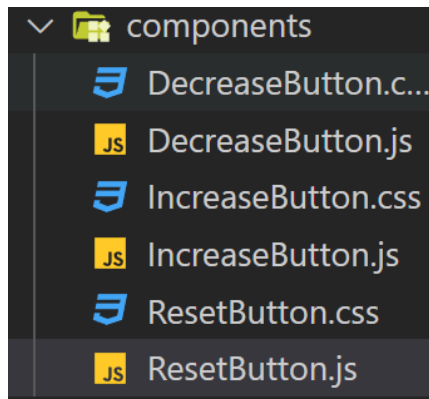
Výsledek na stránce



## 42. React - Resetovací tlačítko

Video: <https://youtu.be/S1BdplkEzWo>

Ve složce COMPONENTS vytvoříme dva nové soubory ResetButton.js a ResetButton.css



### App.js

Naimportujeme tlačítko RESETBUTTON, které vytvoříme v souboru ResetButton.js. Použijeme tlačítko v return a pošleme funkci resetToZero do komponenty RESETBUTON pod názvem RESET.

```
import DecreaseButton from "../components/DecreaseButton"
import IncreaseButton from "../components/IncreaseButton"
import ResetButton from "../components/ResetButton"
import { useState } from "react"

const App = () => {
  const [value, setValue] = useState(0)

  const decreaseOne = () => {
    const newValue = value - 1
    setValue(newValue)
  }

  const increaseOne = () => {
    const newValue = value + 1
    setValue(newValue)
  }

  const resetToZero = () => {
    setValue(0)
  }

  return (
    <div>
      <h1>Počítadlo</h1>
```

```

    <h2>{value}</h2>
    <DecreaseButton decrease={decreaseOne}/>
    <IncreaseButton increase={increaseOne}/>
    <ResetButton reset={resetToZero}/>
  </div>
)
}

export default App

```

### ResetButton.js

Pomocí PROPS zavoláme po kliknutí RESET, který si posíláme z App.js

```

import './ResetButton.css'

const ResetButton = (props) => {
  return (
    <button onClick={props.reset}>Reset na 0</button>
  )
}

export default ResetButton

```

## 43. React - Stylování aplikace

Video: <https://youtu.be/C6PYw8aojts>

**App.js** - přidání classy

```
return (  
  <div className="counter">  
    <h1>Počítadlo</h1>  
    <h2>{value}</h2>
```

**Přidání classy BTN do všech tlačítek**

**DecreaseButton.js**

```
const DecreaseButton = (props) => {  
  return (  
    <button className="btn" onClick={props.decrease}>Snížit o  
1</button>  
  )  
}
```

**IncreaseButton.js**

```
const IncreaseButton = (props) => {  
  return (  
    <button className="btn" onClick={props.increase}>Zvýšit o  
1</button>  
  )  
}
```

**ResetButton.js**

```
const ResetButton = (props) => {  
  return (  
    <button className="btn" onClick={props.reset}>Reset na  
0</button>  
  )  
}
```

## Stylování

index.css

Všechno stylování provádíme v index.css

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
  
.counter {  
  width: 100%;  
  min-height: 100vh;  
  background-color: rgb(6, 74, 80);  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
}  
  
.counter h1 {  
  color: white;  
  font-size: 40px;  
}  
  
.counter h2 {  
  color: white;  
  font-size: 40px;  
  margin-bottom: 10px;  
}  
  
.btn {  
  background-color: rgb(3, 33, 19);  
  color: white;  
  padding: 5px 10px;  
  border: none;  
  border-radius: 5px;  
  margin-top: 5px;  
}
```

Výsledek na stránce

# Počítadlo

0

Snížit o 1

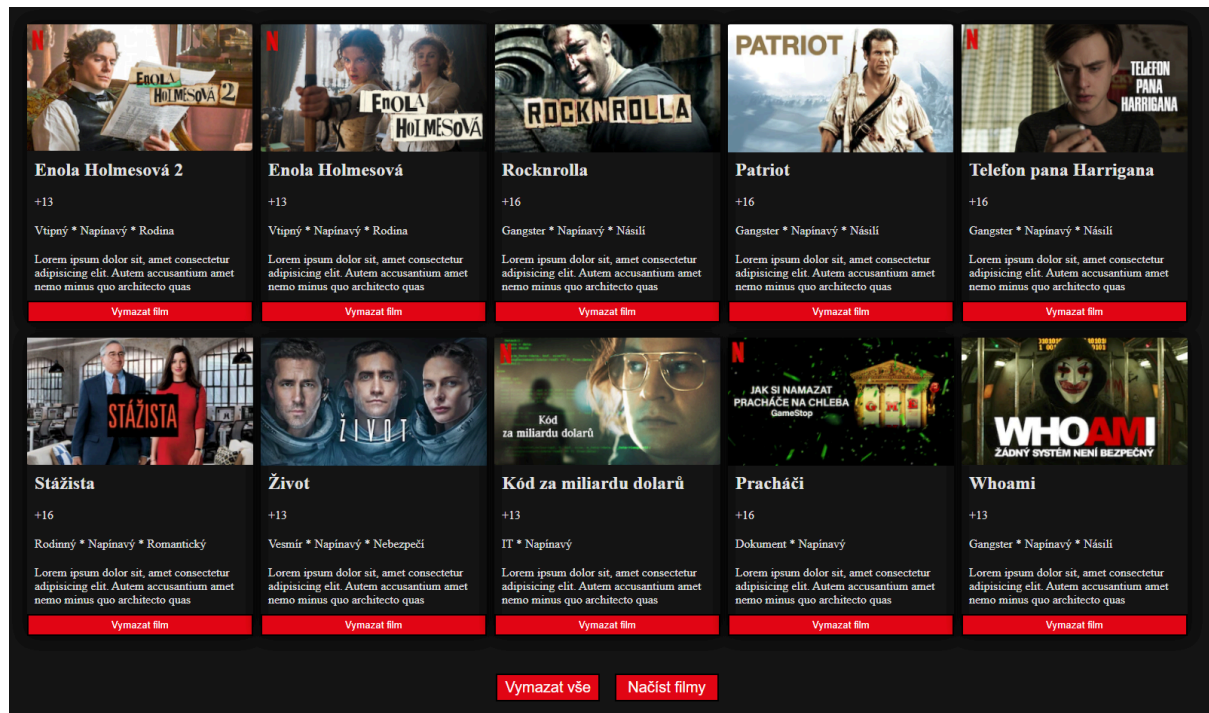
Zvýšit o 1

Reset na 0

## 44. React - Ukázka naší Netflix aplikace

Video: <https://youtu.be/K8wupj9NMJ8>

Takto bude vypadat naše aplikace



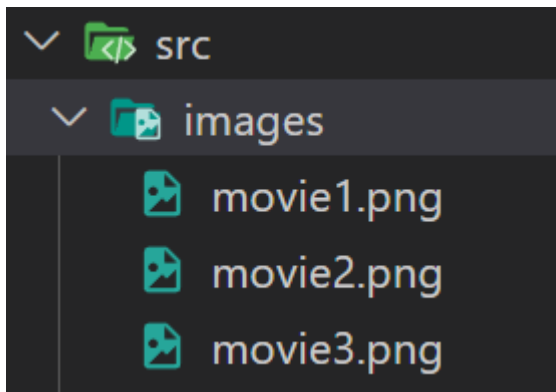
## 45. React - Založení projektu a stažení obrázků

Video: <https://youtu.be/EkUu-J-Af6Q>

Obrázky do složky images

<https://drive.google.com/drive/folders/1td236jZt7gvk1WxD2YzB9Edot7AxMk2P?usp=sharing>

Ve složce SRC si vytvoříme složku IMAGES a do ní vložíme obrázky z výše uvedeného odkazu.



Index.html

```
<body>
  <div id="result"></div>
</body>
```

Index.js

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

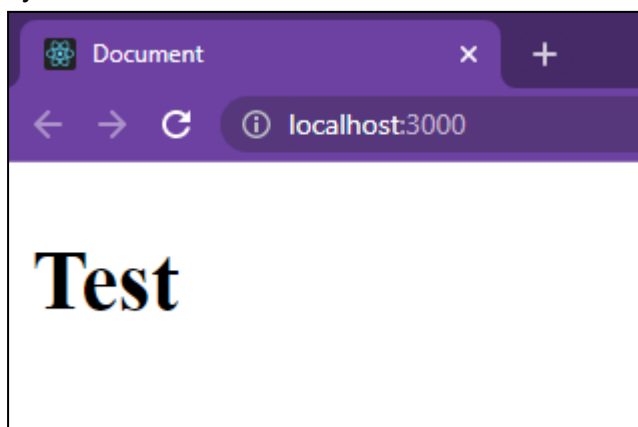
const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

App.js

```
const App = () => {
  return (
    <h2>Test</h2>
  )
}

export default App
```

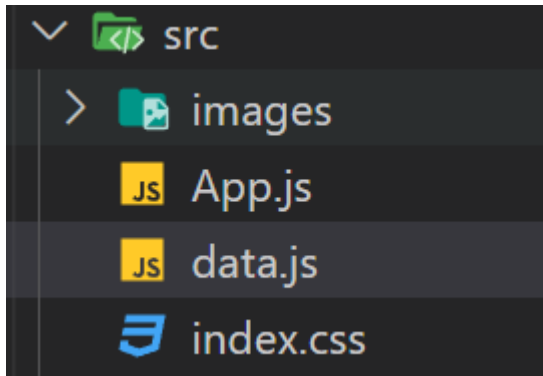
Výsledek na stránce



## 46. React - Vkládáme data a natahujeme obrázky

Video: <https://youtu.be/WNNnZnbtyTQ>

Ve složce SRC vytvoříme soubor DATA.JS.



### data.js

Do data.js zkopírujeme toto pole objektů, kde každý objekt představuje jeden film. Pozor na poslední řádek s exportem, který v souboru potřebujeme.

```
const allMovies = [  
  {  
    id: 1,  
    image: image1,  
    title: "Enola Holmesová 2",  
    age: "+13",  
    tags: "Vtipný * Napínavý * Rodina",  
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem  
accusantium amet nemo minus quo architecto quas",  
  },  
  {  
    id: 2,  
    image: image2,  
    title: "Enola Holmesová",  
    age: "+13",  
    tags: "Vtipný * Napínavý * Rodina",  
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem  
accusantium amet nemo minus quo architecto quas",  
  },  
  {  
    id: 3,  
    image: image3,  
    title: "Rocknrolla",  
    age: "+16",  
    tags: "Gangster * Napínavý * Násilí",  
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem  
accusantium amet nemo minus quo architecto quas",  
  },  
  {  
    id: 4,  
    image: image4,  
    title: "Patriot",  
    age: "+16",  
  },  
]
```

```

        tags: "Gangster * Napínový * Násilí",
        description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem
accusantium amet nemo minus quo architecto quas",
    },
    {
        id: 5,
        image: image5,
        title: "Telefon pana Harrigana",
        age: "+16",
        tags: "Gangster * Napínový * Násilí",
        description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem
accusantium amet nemo minus quo architecto quas",
    },
    {
        id: 6,
        image: image6,
        title: "Stážista",
        age: "+16",
        tags: "Rodinný * Napínový * Romantický",
        description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem
accusantium amet nemo minus quo architecto quas",
    },
    {
        id: 7,
        image: image7,
        title: "Život",
        age: "+13",
        tags: "Vesmír * Napínový * Nebezpečí",
        description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem
accusantium amet nemo minus quo architecto quas",
    },
    {
        id: 8,
        image: image8,
        title: "Kód za miliardu dolarů",
        age: "+13",
        tags: "IT * Napínový",
        description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem
accusantium amet nemo minus quo architecto quas",
    },
    {
        id: 9,
        image: image9,
        title: "Pracháči",
        age: "+16",
        tags: "Dokument * Napínový",
        description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem
accusantium amet nemo minus quo architecto quas",
    },
    {
        id: 10,
        image: image10,
        title: "Whoami",

```

```
    age: "+13",
    tags: "Gangster * Napínavý * Násilí",
    description: "Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium amet nemo minus quo architecto quas",
  },
]

export default allMovies
```

### data.js

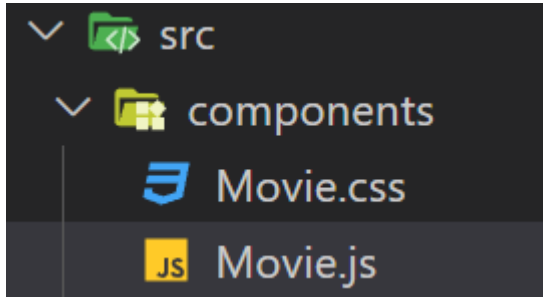
Nahoru do data.js naimportujeme všechny potřebné obrázky ze složky IMAGES.

```
import image1 from "../images/movie1.png"
import image2 from "../images/movie2.png"
import image3 from "../images/movie3.png"
import image4 from "../images/movie4.png"
import image5 from "../images/movie5.png"
import image6 from "../images/movie6.png"
import image7 from "../images/movie7.png"
import image8 from "../images/movie8.png"
import image9 from "../images/movie9.png"
import image10 from "../images/movie10.png"
```

## 47. React - Vypisujeme filmy do stránky

Video: <https://youtu.be/JZ7lp5O8lkU>

Ve složce SRC si vytvoříme složku COMPONENTS. V ní si vytvoříme dva soubory Movie.js a Movie.css.



### Movie.js

Vytvoříme komponentu, kam nainportujeme data z data.js. Data vypíšeme do stránky pomocí funkce MAP. Nezapomeňte, že každý výpis filmu by měl mít speciální identifikátor (klíč) a k tomu využíváme ID (viz. žlutá část kódu).

```
import "../Movie.css"
import data from "../data"

const Movie = () => {
  return <div>
    {
      data.map( (oneMovie) => {
        const {id, image, title, age, tags, description} = oneMovie

        return <div key={id}>
          <img src={image} alt="" />
          <h2>{title}</h2>
          <p>{age}</p>
          <p>{tags}</p>
          <p>{description}</p>
        </div>
      })
    }
  </div>
}

export default Movie
```

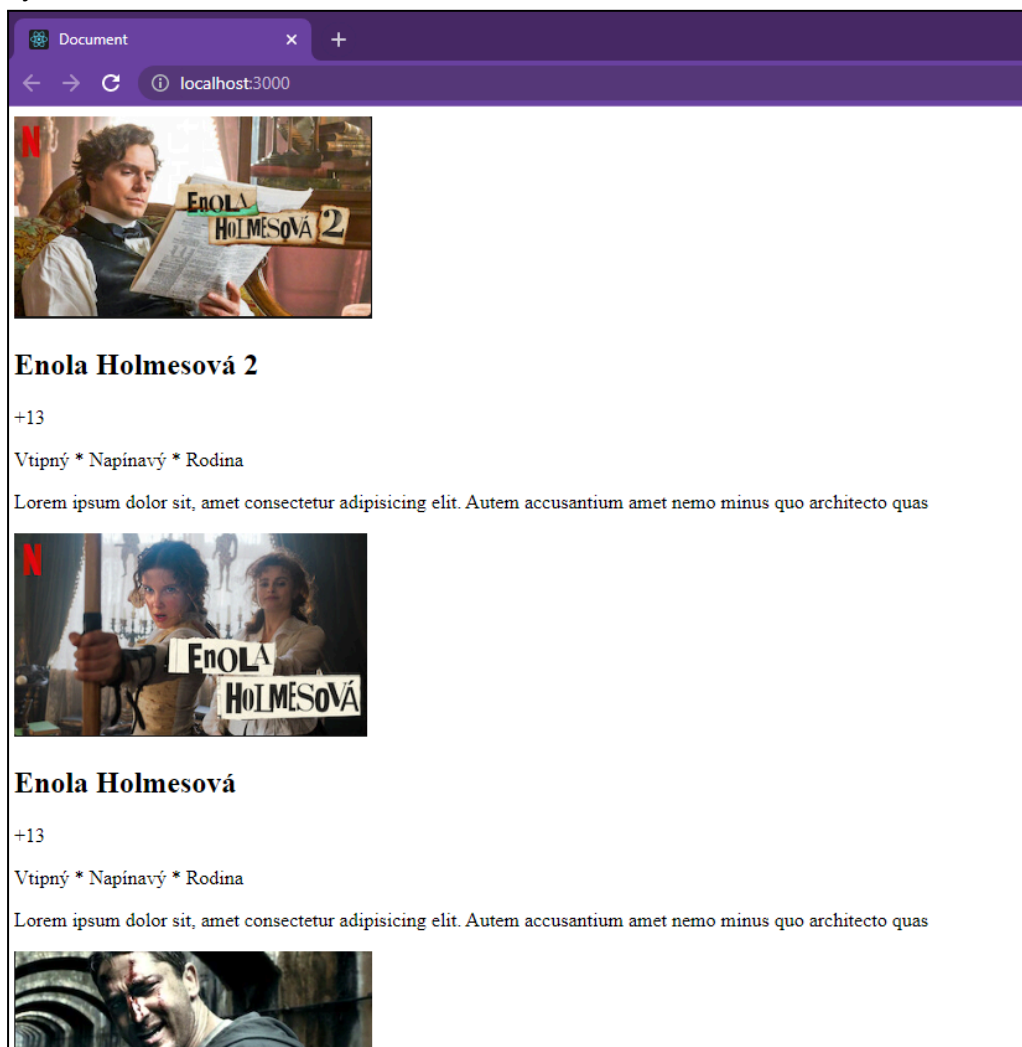
## App.js

Do App.js naimportujeme komponentu MOVIE a umístíme ji do return.

```
import Movie from "../components/Movie"
```

```
const App = () => {  
  return (  
    <Movie />  
  )  
}  
  
export default App
```

## Výsledek na stránce



## 48. React - Stylování stránky a filmů

Video: <https://youtu.be/zsLxdCs1WfU>

### Movie.js

přidáme classu all-movies

```
const Movie = () => {  
  return <div className="all-movies">  
    {  
      data.map( (oneMovie) => {
```

přidáme classu one-movie

```
      return <div className="one-movie" key={id}>  
        <img src={image} alt="" />  
        <h2>{title}</h2>
```

### index.css

```
body {  
  background-color: #141414;  
}
```

### Movie.css

```
.all-movies {  
  width: 90%;  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
  flex-wrap: wrap;  
  margin: 40px auto;  
}  
  
.one-movie {  
  width: 300px;  
  box-shadow: 0px 0px 20px black;  
  color: #e5e5e5;  
  margin: 10px 5px;  
  display: flex;  
  flex-direction: column;  
  transition: all 0.5s linear;  
}  
  
.one-movie:hover {  
  transform: scale(1.04);  
}  
  
.one-movie img {
```

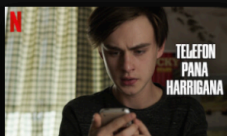
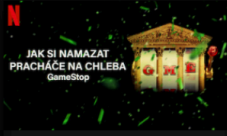
```

    height: 170px;
}

.one-movie h2,
.one-movie p {
    padding-left: 10px;
    margin-top: 10px;
    margin-bottom: 10px;
}

```

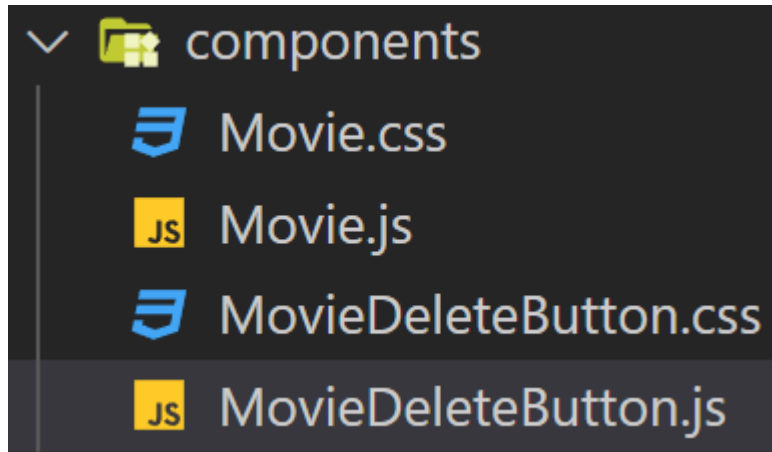
## Výsledná podoba na stránkách

|                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <p><b>Enola Holmesová 2</b></p> <p>+13</p> <p>Vtipný * Napínávací * Rodina</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p> |  <p><b>Enola Holmesová</b></p> <p>+13</p> <p>Vtipný * Napínávací * Rodina</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p> |  <p><b>Rocknrolla</b></p> <p>+16</p> <p>Gangster * Napínávací * Násilí</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p> |  <p><b>Patriot</b></p> <p>+16</p> <p>Gangster * Napínávací * Násilí</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p> |  <p><b>Telefon pana Harrigana</b></p> <p>+16</p> <p>Gangster * Napínávací * Násilí</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p> |
|  <p><b>Stážísta</b></p> <p>+16</p> <p>Rodinný * Napínávací * Romantický</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p>    |  <p><b>Život</b></p> <p>+13</p> <p>Vesmír * Napínávací * Nebezpečí</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p>       |  <p><b>Kód za miliardu dolarů</b></p> <p>+13</p> <p>IT * Napínávací</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p>   |  <p><b>Pracháči</b></p> <p>+16</p> <p>Dokument * Napínávací</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p>        |  <p><b>Whoami</b></p> <p>+13</p> <p>Gangster * Napínávací * Násilí</p> <p>Lorem ipsum dolor sit, amet consectetur adipiscing elit. Autem accusantium amet nemo minus quo architecto quas</p>                |

## 49. React - Tlačítko na vymazání filmů a useState

Video: <https://youtu.be/GKAIKwVPcPI>

Ve složce COMPONENTS si založíme další dva soubory MovieDeleteButton.js a MovieDeleteButton.css



### Movie.js

Naimportujeme do Movie.js komponentu MOVIEDELETEBUTTON a přidáme ji na do return, aby se vyrendrovala (vypsala) ke každému filmu.

Poté naimportujeme useState, které použijeme k profilování všech filmů a vymažeme film podle jeho ID. Poté pomocí setMovieList nastavíme vyfiltrované filmy z FILTEREDMOVIES do proměnné movieList a useState zabezpečí, že se opět vše přepíše (přerendruje). Takže se změny promítnou do stránky.

```
import "../Movie.css"
import data from "../data"
import MovieDeleteButton from "../MovieDeleteButton"

const Movie = () => {
  const [movieList, setMovieList] = useState(data)

  const deleteOneMovie = (idecko) => {
    const filteredMovies = movieList.filter( (oneMovie) => {
      return oneMovie.id !== idecko
    })
    setMovieList(filteredMovies)
  }

  return <div className="all-movies">
    {
      movieList.map( (oneMovie) => {
```

```

        const {id, image, title, age, tags,
description} = oneMovie

        return <div className="one-movie" key={id}>
            <img src={image} alt="" />
            <h2>{title}</h2>
            <p>{age}</p>
            <p>{tags}</p>
            <p>{description}</p>
            <MovieDeleteButton deleteMovie={() =>
deleteOneMovie(id)} />

            </div>

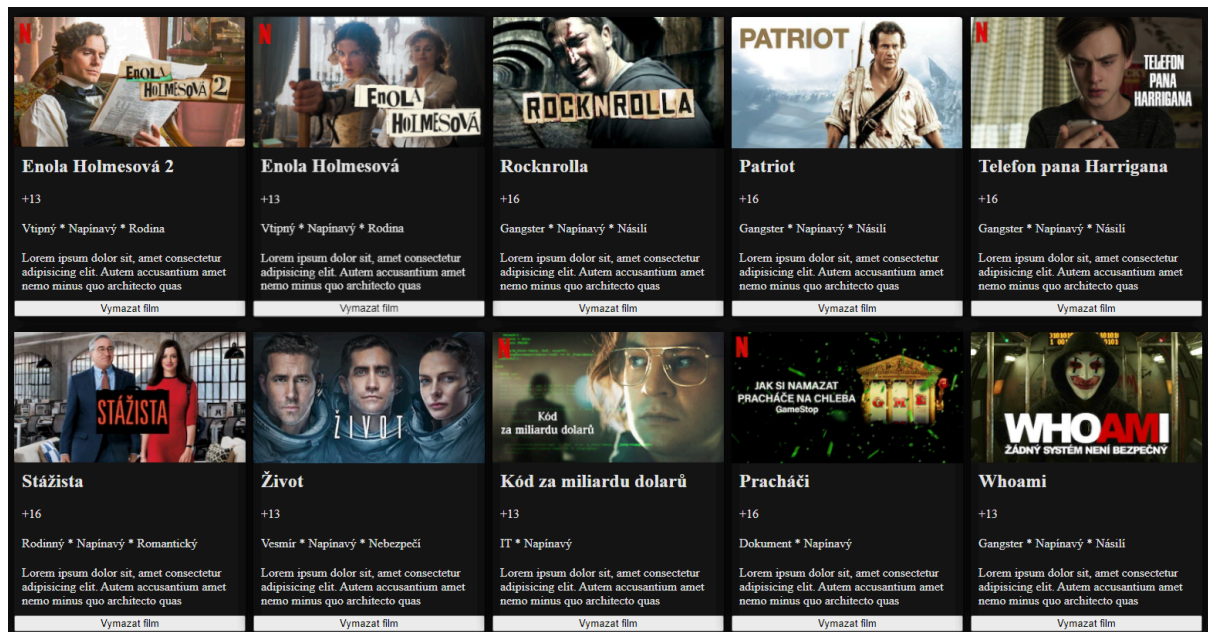
        ))
    }

</div>
}

export default Movie

```

## Výsledek na stránce



## 50. React - Stylujeme tlačítko na vymazání filmu

Video: [https://youtu.be/tOVg3Mjkh\\_4](https://youtu.be/tOVg3Mjkh_4)

MovieDeleteButton.js

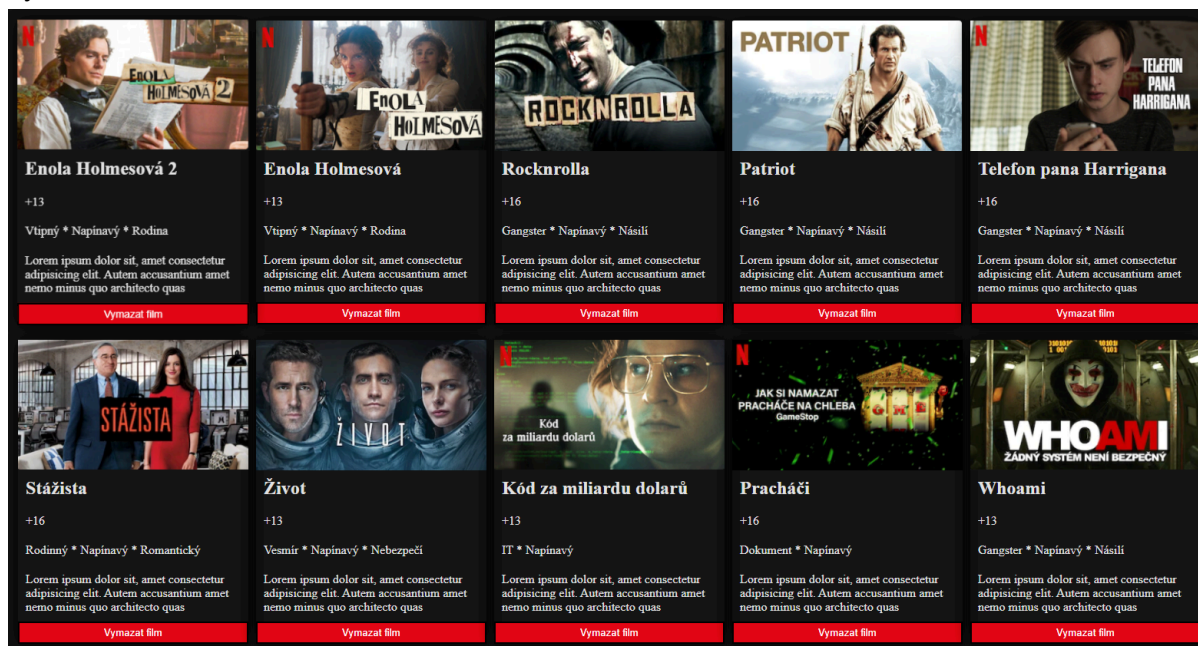
Přidáme classu movie-delete-button

```
return (  
  <button className="movie-delete-button"  
    onClick={props.deleteMovie}>Vymazat film</button>  
)
```

MovieDeleteButton.css

```
.movie-delete-button {  
  color: white;  
  background-color: #e50914;  
  padding: 5px;  
  cursor: pointer;  
}
```

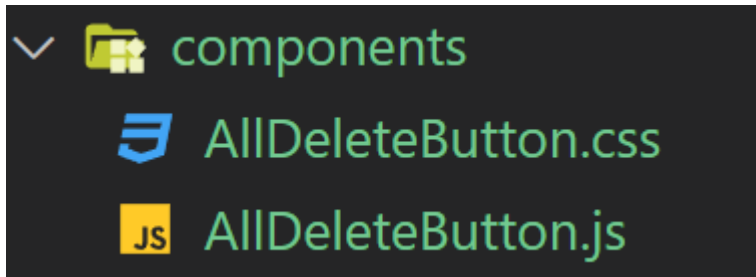
Výsledek na stránce



## 51. React - Tlačítko na vymazání všech filmů

Video: [https://youtu.be/krV1\\_QCRxml](https://youtu.be/krV1_QCRxml)

Založíme si ve složce COMPONENTS další dva soubory AllDeleteButton.js a AllDeleteButton.css



### AllDeleteButton.js

Vytvoříme komponentu, která bude obsahovat tlačítko. Jakmile se na tlačítko klikne, tak se přes props vyhledá v nadřazeném souboru (Movie.js) props deleteMovies.

```
import "../AllDeleteButton.css"

const AllDeleteButton = (props) => {
  return (
    <button className="main-delete-button"
    onClick={props.deleteMovies}>Vymazat vše</button>
  )
}

export default AllDeleteButton
```

### Movie.js

Zde AllDeleteButton.js naimportujeme.

```
import MovieDeleteButton from "../MovieDeleteButton"
import AllDeleteButton from "../AllDeleteButton"
import { useState } from "react"
```

Vytvoříme si funkci, která nastaví movieList na prázdné pole.

```
const deleteAllMovies = () => {
  setMovieList([])
}
```

Přes props pošleme funkci DELETEALLMOVIES pod názvem DELETEMOVIES do komponenty AllDeleteButton, kde se bude spouštět po kliknutí (viz předchozí kód v AllDeleteButton.js)

```
return <section>
  <div className="all-movies">
    {
      movieList.map( (oneMovie) => {
```

```

        const {id, image, title, age, tags,
description} = oneMovie

        return <div className="one-movie" key={id}>
            <img src={image} alt="" />
            <h2>{title}</h2>
            <p>{age}</p>
            <p>{tags}</p>
            <p>{description}</p>
            <MovieDeleteButton deleteMovie={ () =>
deleteOneMovie(id) }/>
        </div>
    ))
}

</div>
<div>
    <AllDeleteButton deleteMovies={deleteAllMovies}/>
</div>
</section>

```

## Stylování

AllDeleteButton.js

Zkontrolujte, že BUTTON v AllDeleteButton.js má classu main-delete-button.

```

return (
    <button className="main-delete-button"
onClick={props.deleteMovies}>Vymazat vše</button>
)

```

AllDeleteButton.css

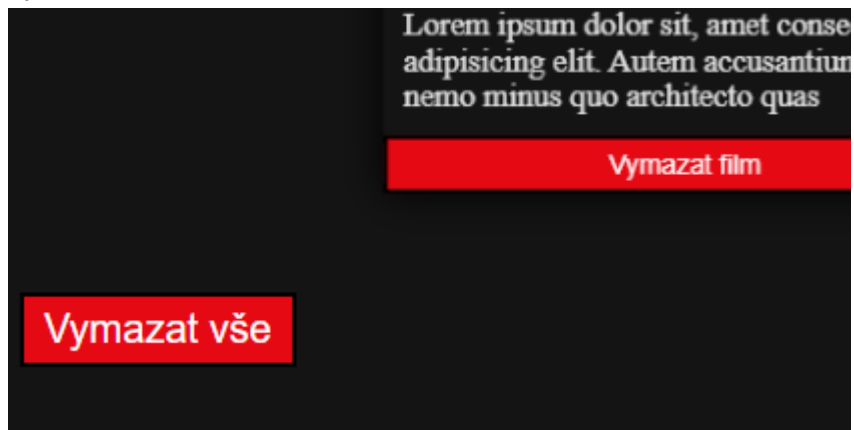
Zde si ji nastylujeme

```

.main-delete-button {
    color: white;
    background-color: #e50914;
    font-size: 20px;
    padding: 5px 10px;
    cursor: pointer;
}

```

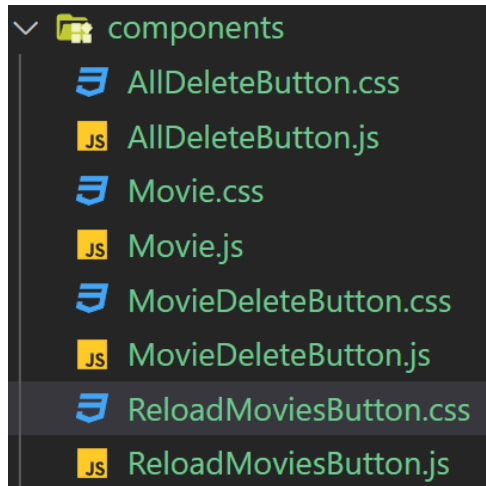
Výsledek na stránce



## 52. React - Tlačítko na opětovné načtení všech filmů

Video: <https://youtu.be/qWaZPfhZz48>

Ve složce COMPONENTS si vytvoříme další dva soubory s názvy ReloadMoviesButton.js a ReloadMoviesButton.css



### ReloadMoviesButton.js

Vytvoříme komponentu, která bude obsahovat tlačítko. Na kliknutí se přes props (v nadřazeném souboru Movie.js) nalezene RELOADMOVIES.

```
import "../ReloadMoviesButton.css"

const ReloadMoviesButton = (props) => {
  return (
    <button className="reload-movies-button"
    onClick={props.reloadMovies}>Načíst filmy</button>
  )
}

export default ReloadMoviesButton
```

### Movie.js

Naimportujeme komponentu ReloadMoviesButton.js.

```
import AllDeleteButton from "../AllDeleteButton"
import ReloadMoviesButton from "../ReloadMoviesButton"
import { useState } from "react"
```

Vytvoříme funkci na nastavení movieList zpět na všech 10 filmů, které jsou uložené v proměnné data.

```
const reloadAllMovies = () => {
  setMovieList(data)
}
```

Přidáme komponentu do return. Přes props s názvem reloadMovies pošleme do ReloadMoviesButton.js funkci RELOADALLMOVIES, kterou jsme si vytvořili v Movie.js.

```
<div>
```

```

        <AllDeleteButton deleteMovies={deleteAllMovies}/>
        <ReloadMoviesButton reloadMovies={reloadAllMovies}/>
    </div>

```

## Stylování

ReloadMoviesButton.js

Zkontrolujeme, zda má button classu reload-movies-button.

```

return (
    <button className="reload-movies-button"
onClick={props.reloadMovies}>Načíst filmy</button>
)

```

ReloadMoviesButton.css

Classu reload-movies-button nastylujeme.

```

.reload-movies-button {
    color: white;
    background-color: #e50914;
    font-size: 20px;
    padding: 5px 15px;
    cursor: pointer;
    margin-left: 20px;
}

```

Výsledek na stránce



## 53. React - Závěrečné stylování Netflix aplikace

Video: <https://youtu.be/bVoyfFq9Esg>

### Movie.js

Přidáme classu button-box, která obaluje obě dvě tlačítka. Tu použijeme v Movie.css ke stylování

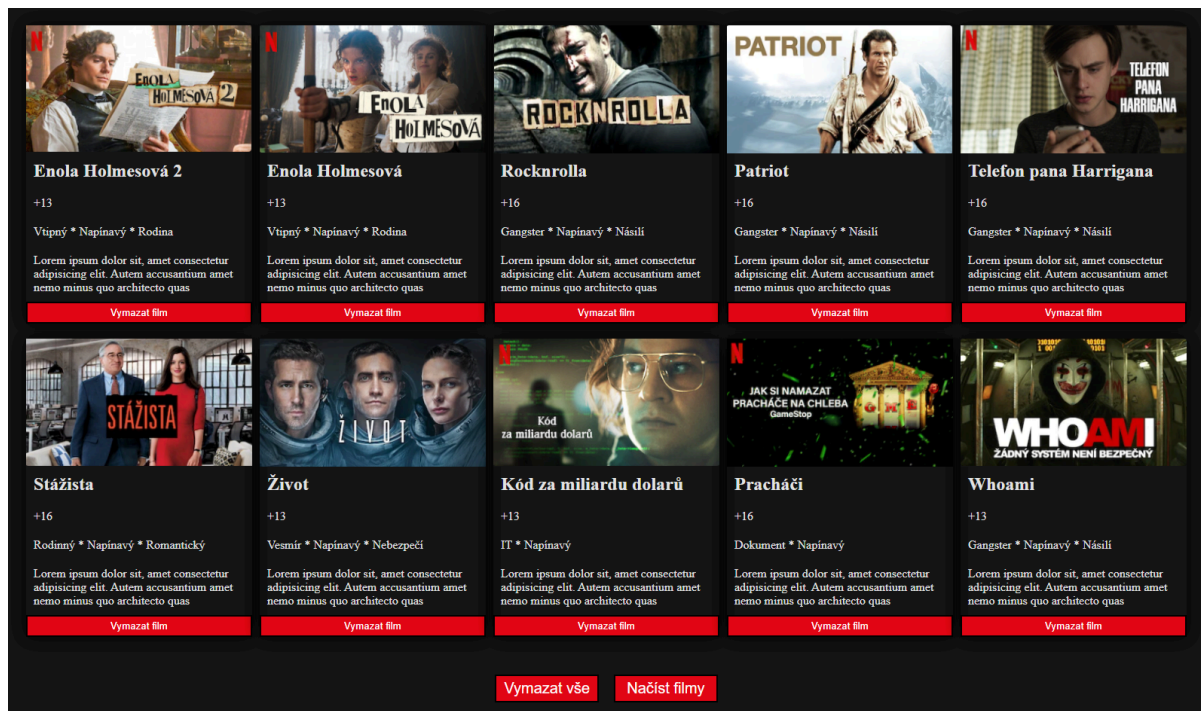
```
<div className="button-box">
  <AllDeleteButton deleteMovies={deleteAllMovies}/>
  <ReloadMoviesButton reloadMovies={reloadAllMovies}/>
</div>
```

### Movie.css

Pomocí flexboxu zarovnáme obě tlačítka na střed

```
.button-box {
  display: flex;
  flex-direction: row;
  justify-content: center;
}
```

### Výsledek na stránce



## 54. React - Založení aplikace

Video: <https://youtu.be/NPy0hm8KV4c>

Index.html

Do index.html vložíme div s ID result

```
<body>
  <div id="result"></div>
</body>
```

Index.js

Klasický obsah index.js, kde se napojuje App.js a index.css. A také načítáme DIV s ID result, do kterého vše budeme umisťovat.

```
import { createRoot } from 'react-dom/client';

import './index.css';
import App from './App';

const container = document.getElementById('result')
const root = createRoot(container)
root.render(<App tab="home" />)
```

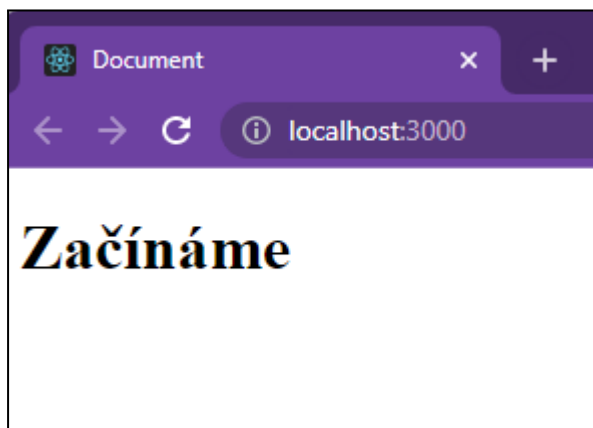
App.js

Klasické vytvoření komponenty

```
const App = () => {
  return (
    <h1>Začínáme</h1>
  )
}

export default App
```

Výsledek na stránce



## 55. React - Ukázka useEffectu

Video: <https://youtu.be/EOoltNj9wCY>

App.js

Abychom mohli useEffect používat, tak ho nejdříve musíme nainportovat. Zde ho importujeme společně s useState.

```
import { useState, useEffect } from "react"

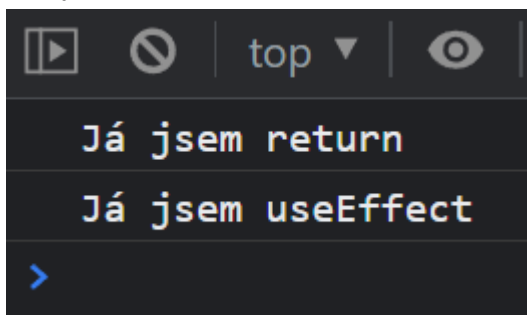
const App = () => {

  useEffect( () => {
    console.log("Já jsem useEffect");
  })

  return (
    <div>
      <h1>Začínáme</h1>
      {console.log("Já jsem return")}
    </div>
  )
}

export default App
```

Výsledek v konzoli - vidíme, že useEffect se spouští až nakonec. Jeho console.log je až druhý



## 56. React - Zkoumáme detailněji useEffect

Video: <https://youtu.be/b3k7ZXhcsVg>

Kliknutí na tlačítko zavolá setValue. Value se nastaví o 1 vyšší a nová hodnota se do value uloží. Tím se spustí znovunačtení stránky (vypsání) díky useState. To opět provede vše v return a poté spustí znovu useEffect.

Závěrem - po každém kliknutí na tlačítko se znovu načte stránka a nakonec se vždy spustí useEffect.

```
import { useState, useEffect } from "react"

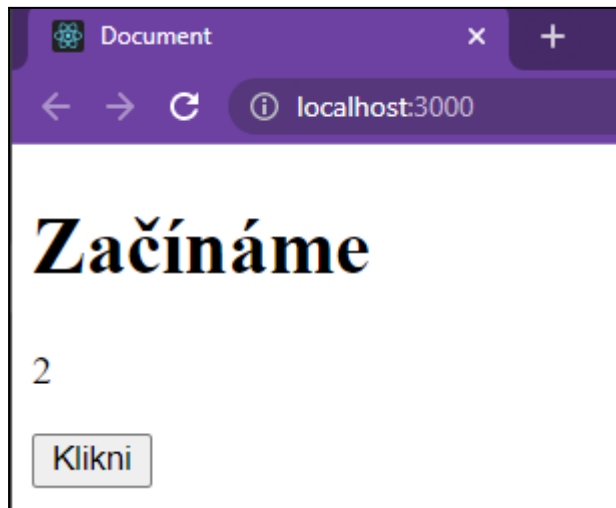
const App = () => {
  const [value, setValue] = useState(0)

  useEffect( () => {
    console.log("Já jsem useEffect");
  })

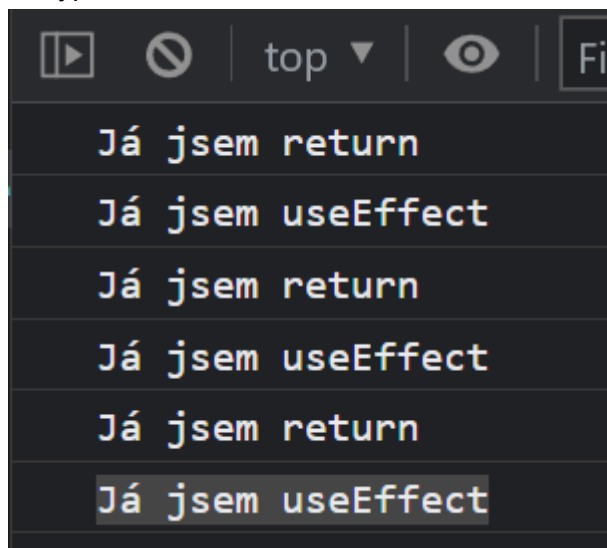
  return (
    <div>
      <h1>Začínáme</h1>
      <p>{value}</p>
      {console.log("Já jsem return")}
      <button onClick={ () => setValue(value + 1)}>Klikni</button>
    </div>
  )
}

export default App
```

Výsledek na stránce po 2 kliknutí na tlačítko Klikni



3x vypsání useEffectu v konzoli



## 57. React - Dokazujeme fungování useState a useEffect pomocí výpisů do konzole

Video: <https://youtu.be/p9pbpoyiO9s>

Dáme do předešlého kódu přidáme více console.logů, aby bylo jasné, co se vypisuje jako první atd. Opět se nám potvrdí, že useEffect vypisuje při každém načtení stránky skutečně jako poslední.

```
import { useState, useEffect } from "react"

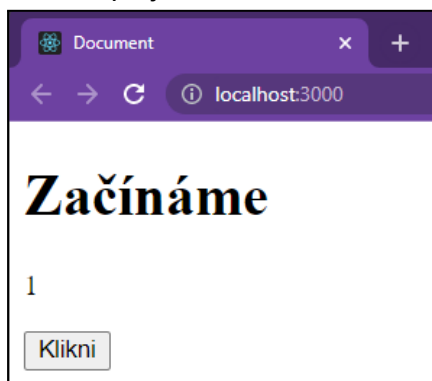
const App = () => {
  console.log("text úplně nahoře")
  const [value, setValue] = useState(0)

  useEffect( () => {
    console.log("Já jsem useEffect")
  })

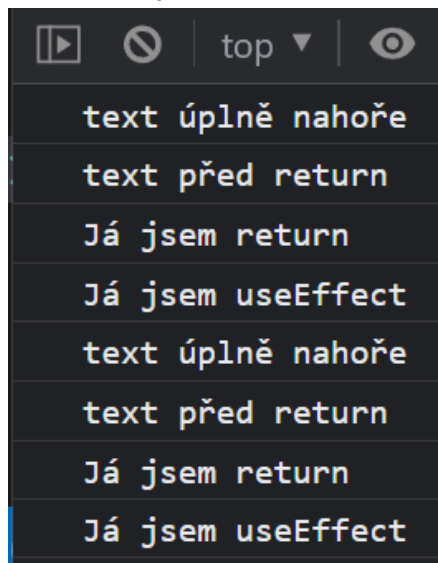
  console.log("text před return")
  return (
    <div>
      <h1>Začínáme</h1>
      <p>{value}</p>
      {console.log("Já jsem return")}
      <button onClick={ () => setValue(value + 1)}>Klikni</button>
    </div>
  )
}

export default App
```

Stránka po jednom kliknutí na tlačítko Klikni



Konzole po jednom kliknutí na tlačítko Klikni



## 58. React - Příklad využití useEffectu

Video: <https://youtu.be/1BP4t8X-Jd4>

Pokud bychom se pokusili načíst tlačítko přes classu BTN a uložit ho do proměnné button, tak se do proměnné uloží null. Bude prázdné. Proč? Protože v danou chvíli, kdy ukládáme tlačítko do proměnné, tak tlačítko ještě neexistuje. Máme možnost kód přesunout do useEffect (viz dva žluté řádky níže) a tím máme zajištěno, že se tlačítko uloží do proměnné ve chvíli, kdy ho budeme znát, protože useEffect se spouští jako poslední.

```
import { useState, useEffect } from "react"

const App = () => {
  const [value, setValue] = useState(0)

  useEffect( () => {
    const button = document.querySelector(".btn")
    button.textContent = `Klik číslo ${value}`
  })

  return (
    <div>
      <h1>Začínáme</h1>
      <p>{value}</p>
      {console.log("Já jsem return")}
      <button className="btn" onClick={ () => setValue(value +
1)}>Klikni</button>
    </div>
  )
}

export default App
```

Výsledek na stránce (pokud nebylo kliknuto)



Výsledek na stránce po prvním kliknutí na tlačítko

# Začínáme

1

Klik číslo 1

## 59. React - useEffect a podmínka

Video: <https://youtu.be/8aPZYSZSFyw>

### App.js

Budeme chtít, aby se po načtení stránky zobrazil v tlačítku text Klikni, ale až po prvním kliknutí, aby se napsalo Klik číslo 1, poté Klik číslo 2 atd. To vyřešíme pomocí podmínky, kterou umístíme do vnitřku useEffectu.

```
useEffect( () => {  
  const button = document.querySelector(".btn")  
  if (value >= 1) {  
    button.textContent = `Klik číslo ${value}`  
  } else {  
    button.textContent = "Klikni"  
  }  
})
```

Výsledek na stránce po načtení stránky



Výsledek na stránce po prvním kliknutí na tlačítko



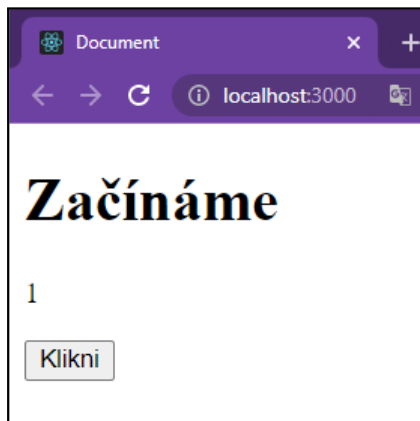
## 60. React - useEffect běží jen jednou

Video: [https://youtu.be/4R\\_0UP59biU](https://youtu.be/4R_0UP59biU)

Pokud chceme, aby se useEffect spustil pouze jednou a ne pokaždé, když dojde k načtení stránky, tak můžeme přidat čárku a závorku před poslední běžnou závorku (viz žluté zvýraznění v kódu)

```
useEffect( () => {  
  const button = document.querySelector(".btn")  
  if (value >= 1) {  
    button.textContent = `Klik číslo ${value}`  
  } else {  
    button.textContent = "Klikni"  
  }  
}, [ ] )
```

Po prvním kliknutí na tlačítko se useEffect nespustí (nezmění se text v tlačítku), protože jsme ho nastavili, aby se spouštěl pouze jednou (to jednou je při prvním načtení stránky).



## 61. React - useEffect a závislé proměnné (dependencies)

Video: <https://youtu.be/WFZ58bVmDj0>

Do hranatých závorek na konci useEffect můžeme psát závislou proměnnou. Ta znamená, že pokud se změní, tak se na konci vypsání stránky načte pouze tento daný useEffect. V kódu vidíme, že první useEffect se změní jen když dojde ke změně proměnné value (po kliknutí na prvním tlačítko). Druhý se změní, když dojde ke změně proměnné test (po kliknutí na druhé tlačítko).

```
const App = () => {
  const [value, setValue] = useState(0)
  const [test, setTest] = useState(0)

  useEffect( () => {
    const button = document.querySelector(".btn")
    if (value >= 1) {
      button.textContent = `Klik číslo ${value}`
    } else {
      button.textContent = "Klikni"
    }
    console.log("První useEffect - Klikni")
  }, [value])

  useEffect( () => {
    document.title = `Nový titulek ${test}`
    console.log("Druhý useEffect - Titulek")
  }, [test])

  return (
    <div>
      <h1>Začínáme</h1>
      <p>{value}</p>
      <button className="btn" onClick={ () => setValue(value +
1)}>Klikni</button>

      <button onClick={ () => setTest(test + 1) }>Titulek</button>
    </div>
  )
}
```

## 62. React - předěláme App

Video: [https://youtu.be/nP9tobU\\_cDA](https://youtu.be/nP9tobU_cDA)

Do proměnné `windowSize` nastavíme jako výchozí hodnotu současnou šířku okna. Jakmile se stránka poprvé načte, tak se spustí `useEffect`, který do okna přidá událost "resize" (změna velikosti okna). Ta se jen nastaví, ale nespustí to funkci `sizeControl`, protože ta se spustí, až když dojde k události změna šířky okna. Ale my okno teď neměníme. Až ho ale změníme, tak se `sizeControl` zavolá (bez ohledu na `useEffect` - ten ji tam jen nastavil), tak se `windowSize` změní pomocí `setWindowSize` a stránka se znovu vypíše. Takže se objeví na stránce aktuální hodnota v tagu `H2`.

```
import { useState, useEffect } from "react"

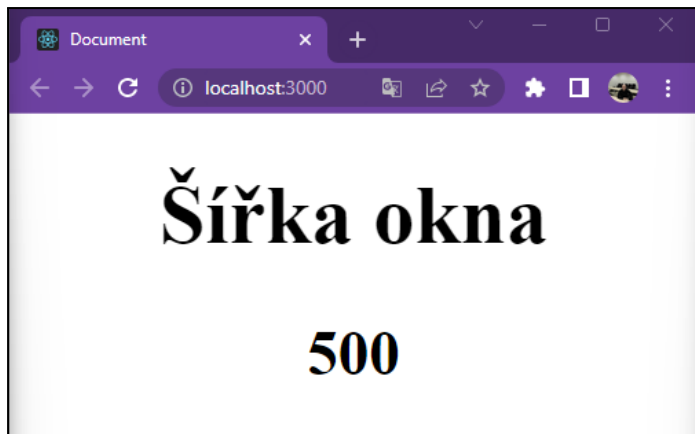
const App = () => {
  const [windowSize, setWindowSize] = useState(window.innerWidth)

  const sizeControl = () => {
    setWindowSize(window.innerWidth)
  }

  useEffect( () => {
    window.addEventListener("resize", sizeControl)
  })

  return (
    <div>
      <h1>Šířka okna</h1>
      <h2>{windowSize}</h2>
    </div>
  )
}

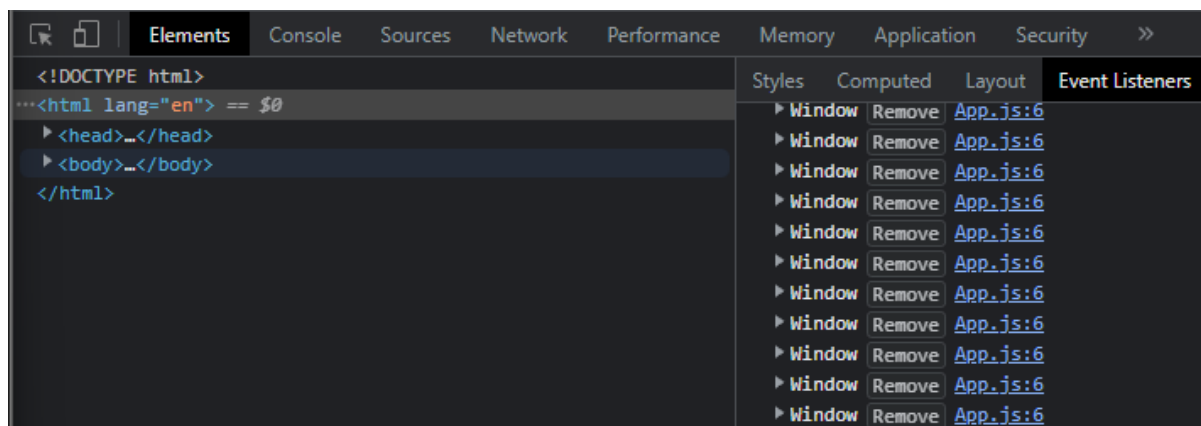
export default App
```



## 63. React - Proč potřebujeme CleanUp function

Video: [https://youtu.be/\\_aVV-roDMjU](https://youtu.be/_aVV-roDMjU)

Náš kód výše funguje tak, že useEffect se s každou změnou okna spouští znovu a znovu. Takže neustále přidává události "resize". To vidíme v konzoli (musíte na levé straně mít označené HTML - viz obrázek). Po rozkliknutí Event listeners vpravo (viz obrázek) a kliknutí na resize vidíme to obrovské množství událostí, které se přidalo. Zde nám to tolik nevadí, ale při složitějších programech by to mohl být problém. A právě to vyřešíme pomocí CleanUp function.



POZOR - zde mi bohatě stačí, že se useEffect spustí jen jednou. Protože při načtení stránky poprvé se na window přidá událost resize a ta spouští funkci sizeControl. A to mi stačí.

```
useEffect( () => {  
    window.addEventListener("resize", sizeControl)  
}, [])
```

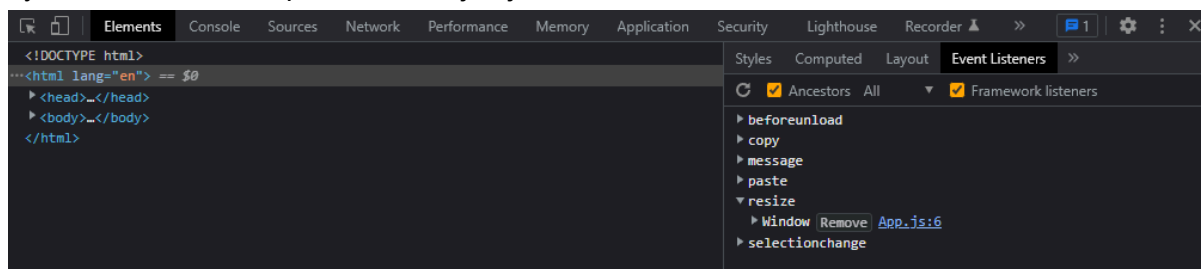
## 64. React - Jak vytvořit cleanUp function

Video: <https://youtu.be/f2heMez3EE4>

CleanUp function řeší problém, který jsme nastínili v předešlém videu. Jak funguje? Stránka se poprvé načte a spustí se useEffect. Ale spustí se jen první dva řádky. Až se stránka znovu vypíše (díky useState), tak se opět spustí useEffect, ale nyní se nejdříve spustí cleanUp function, která je vreturn. Ta odstraní událost a pak se hned spustí první dva řádky, které zase událost přidají. A takto stále dokola. Proto ve výsledku vidíme v konzoli jen jednu událost resize.

```
useEffect( () => {  
  console.log("Já jsem useEffect")  
  window.addEventListener("resize", sizeControl)  
  return () => {  
    console.log("já jsem cleanUp function")  
    window.removeEventListener("resize", sizeControl)  
  }  
})
```

Výsledek v konzoli - vpravo vidíme jen jednu událost u resize



## 65. React - useState, useEffect a API

Video: <https://youtu.be/1CN8aK0DrdE>

Použité API najdeme zde:

<https://kanye.rest/>

Adresa pro získání citátu je zde:

<https://api.kanye.rest/>

Do proměnné URL uložíme výše uvedenou adresu, na které je zobrazený citát. Vytvoříme funkci pro získání dat z url. Nesmíme zapomenout, že data se posílají ve formátu JSON (úsporný textový formát). My musíme kód převést z JSON zpět na výchozí objekt (a uložíme ho do DATA). Výsledkem kódu je, že po opětovném načtení stránky se nám do konzole vypíše nový citát

```
const App = () => {
  const url = "https://api.kanye.rest/"

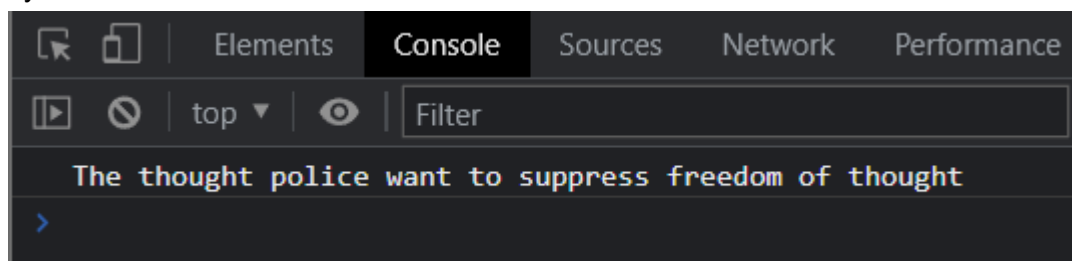
  const getQuote = async () => {
    const response = await fetch(url)
    const data = await response.json()
    console.log(data["quote"])
  }

  getQuote()

  return (
    <h1>Nic</h1>
  )
}

export default App
```

Výsledek v konzoli



## 66. React - Vypsání pomocí proměnné nefunguje

Video: <https://youtu.be/SysRdmue1vM>

Co kdybychom citát přeložili do proměnné a tu vypsali do stránky? To nám nebude fungovat, protože podle console logů uvidíme, že se nejdříve vypíše “výchozí text”, pak “text v return” a až poté “text ve funkci”. Takže funkce se provádí až naposledy. Jenže funkce nastavuje proměnnou QUOTE na nový citát. Potřebujeme něco, co nastaví proměnnou quote a vyrenderuje (vypíše) znovu kód do stránky. A to právě dělá useState.

```
const App = () => {
  const url = "https://api.kanye.rest/"
  let quote = "výchozí text"

  console.log("text nad funkcí");

  const getQuote = async () => {
    const response = await fetch(url)
    const data = await response.json()
    quote = data["quote"]
    console.log("text ve funkci");
  }

  getQuote()

  return (
    <div>
      <h1>{quote}</h1>
      {console.log("text v return")}
    </div>
  )
}

export default App
```

## 67. React - Přidáváme useState

Video: <https://youtu.be/AFzPRESaKag>

Naimportovali jsme useState a nastavili hodnotu QUOTE na výchozí text. Ten jsme vypsalí do stránky a poté se spustila funkce getQuote. Součástí getQuote je ale setQuote a to nastaví proměnnou QUOTE na novou hodnotu (nový citát), která je v proměnné finalQuote. A nyní se znovu vyrenderuje (vypíše) stránka. Ale to opět spustí funkci getQuote a vše jede znovu. Takže jsme vytvořili nekonečný cyklus. **Pokud odkomentujete setQuote(finalquote), tak se spustí nekonečný cyklus, který neustále ve stránce mění citáty.**

```
import { useState } from "react"

const App = () => {
  const [quote, setQuote] = useState("výchozí text")

  const url = "https://api.kanye.rest/"

  const getQuote = async () => {
    const response = await fetch(url)
    const data = await response.json()
    const finalQuote = data["quote"]
    // setQuote(finalQuote)
  }

  getQuote()

  return (
    <div>
      <h1>{quote}</h1>
    </div>
  )
}

export default App
```

## 68. React - useState a useEffect dohromady řeší náš problém

Video: <https://youtu.be/lo-yrA-JnDc>

Přidáním žlutého kódu docílíme toho, že `getQuote` se spustí pouze jednou. To znamená, že tato funkce nastaví pomocí `setQuote` proměnnou `quote` na nový citát a `useState` zabezpečí, že dojde ke znovu vypsání stránky (na stránce se objeví nový citát). A tím vše končí, protože `useEffect` se spustil pouze jednou (jinak řečeno, `getQuote` se spustila pouze jednou).

```
import { useState, useEffect } from "react"

const App = () => {
  const [quote, setQuote] = useState("výchozí text")

  const url = "https://api.kanye.rest/"

  const getQuote = async () => {
    const response = await fetch(url)
    const data = await response.json()
    const finalQuote = data["quote"]
    setQuote(finalQuote)
  }

  useEffect(() => {
    getQuote()
  }, [])

  return (
    <div>
      <h1>{quote}</h1>
    </div>
  )
}

export default App
```

## 69. React - API pro mezinárodního vesmírnou stanici

Video: <https://youtu.be/uoKkuAJk1-k>

International space station API:

<http://open-notify.org/Open-Notify-API/ISS-Location-Now/>

data jsou na této adrese: <http://api.open-notify.org/iss-now.json>

Adresu, ze které chceme získávat data, uložíme do proměnné URL.

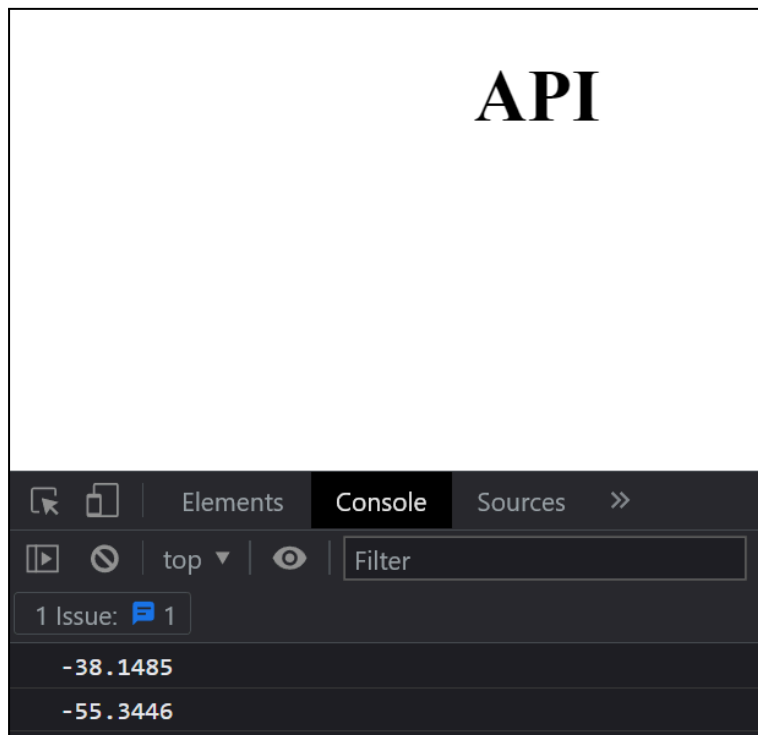
Vytvoříme funkci, která bude data načítat. Vyšleme pomocí fetch požadavek na danou url (žádost o data = request) a zpět se nám vrátí odpověď response a tu uložíme do proměnné RESPONSE.

Odpověď přišla ve formátu JSON, takže ji musíme z JSONu převést na původní formát (objekt). To uděláme pomocí funkce json() a výsledek uložíme do proměnné DATA.

Poté už jen z proměnné DATA vyndáme zeměpisnou šířku (latitude) a délku (longitude).

```
const App = () => {  
  const url = "http://api.open-notify.org/iss-now.json"  
  
  const getCoordinates = async() => {  
    const response = await fetch(url)  
    const data = await response.json()  
    console.log(data["iss_position"]["latitude"])  
    console.log(data["iss_position"]["longitude"])  
  }  
  
  getCoordinates()  
  
  return (  
    <h1>API</h1>  
  )  
}  
  
export default App
```

V konzoli vidíme vypsanou zeměpisnou šířku a délku.



## 70. React - Vypsání zeměpisné šířky a délky ISS do stránky nefunguje

Video: <https://youtu.be/GdzDU5pXAFY>

Dobrá myšlenka (i když nefunkční) je, že se do proměnných latitude a longitude uloží prázdný string.

Ve funkci do těchto proměnných načteme aktuální šířku a délku. A vše vypíšeme do return - tedy do stránky.

Toto ale nebude fungovat kvůli pořadí. Nejdříve se longitude a latitude nastaví na prázdné stringy. Pak se vše vypíše do stránky a pak se zavolá funkce (jak jsme viděli v předcházejících videích). Takže do stránky se vypíší prázdné stringy.

```
const App = () => {
  const url = "http://api.open-notify.org/iss-now.json"
  let latitude = ""
  let longitude = ""

  const getCoordinates = async() => {
    const response = await fetch(url)
    const data = await response.json()
    console.log(data["iss_position"]["latitude"])
    console.log(data["iss_position"]["longitude"])
    latitude = data["iss_position"]["latitude"]
    longitude = data["iss_position"]["longitude"]
  }

  getCoordinates()

  return (
    <div>
      <h1>API</h1>
      <h2>Zeměpisná šířka</h2>
      <p>{latitude}</p>
      <h2>Zeměpisná délka</h2>
      <p>{longitude}</p>
    </div>
  )
}

export default App
```

**API**

**Zeměpisná šířku**

**Zeměpisná délka**

## 71. React - Vypsání, které funguje (useState, useEffect)

Video: <https://youtu.be/ixbUGa00uDs>

Naimportujeme si useState a useEffect, abychom je mohli použít.

Pomocí useState nastavíme proměnné LONGITUDE a LATITUDE na prázdné stringy. Ty se sice vypíší do stránky, ale pak se zavolá funkce setLatitude a setLongitude, které nastavují hodnoty proměnných LATITUDE a LONGITUDE. A také zabezpečí, že se komponenta znovu vypíše do stránky.

Aby nevznikal nekonečný cyklus, tak necháme funkci getCoordinates zavolat pouze jednou až na konci. To zabezpečí useEffect.

```
import { useState, useEffect } from "react"

const App = () => {
  const url = "http://api.open-notify.org/iss-now.json"
  const [latitude, setLatitude] = useState("")
  const [longitude, setLongitude] = useState("")

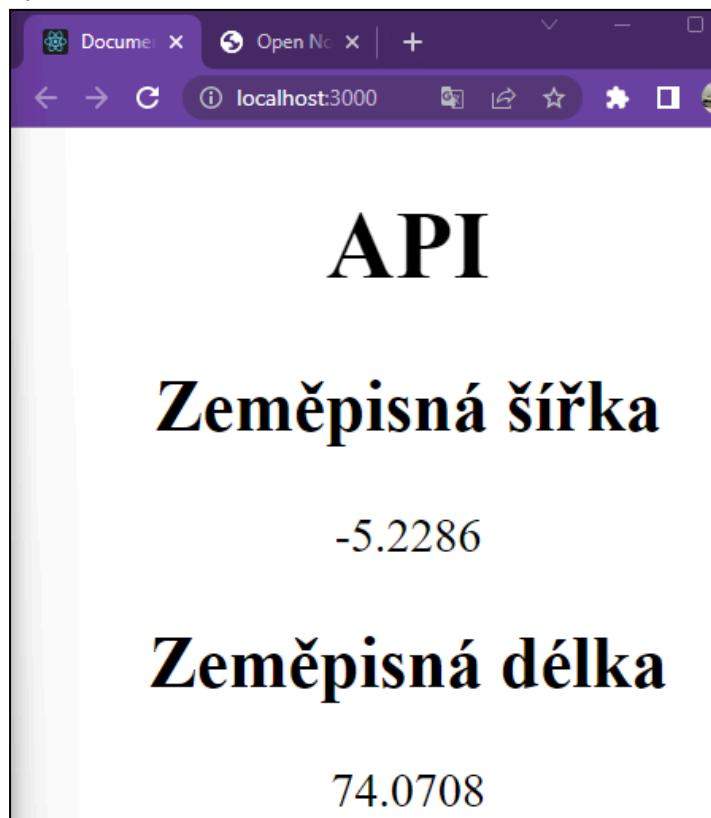
  const getCoordinates = async() => {
    const response = await fetch(url)
    const data = await response.json()
    setLatitude(data["iss_position"]["latitude"])
    setLongitude(data["iss_position"]["longitude"])
  }

  useEffect( () => {
    getCoordinates()
  }, [])

  return (
    <div>
      <h1>API</h1>
      <h2>Zeměpisná šířka</h2>
      <p>{latitude}</p>
      <h2>Zeměpisná délka</h2>
      <p>{longitude}</p>
    </div>
  )
}

export default App
```

Výsledek na stránce



## 72. React - Vylepšujeme stránku o odkaz do map s aktuální pozicí ISS

Video: <https://youtu.be/EgrybAVBt54>

Mapy.cz nám vrací jako úvodní URL toto:

<https://mapy.cz/zakladni?x=14.4701477&y=48.9883169&z=11>

x je zeměpisná délka

y je zeměpisná šířka

z je zoom (přiblížení mapy)

Vytvoříme proměnnou urlMap, kterou pomocí useState nastavíme na prázdný string. Poté ve funkci getCoordinates pomocí setUrlMap nastavíme proměnnou urlMap na hodnotu adresy v mapách, ale doplněnou o současné souřadnice mezinárodní vesmírné stanice. Ta se nastaví do proměnné urlMap a protože je to pomocí useState, tak se komponenta opět vypíše do stránky. My ji vypisujeme do odkazu A, které má referenci (href), která udává, kam má odkaz směřovat. Odkaz má směřovat na adresu, která je uložena v urlMap.

```
import { useState, useEffect } from "react"

const App = () => {
  const url = "http://api.open-notify.org/iss-now.json"
  const [latitude, setLatitude] = useState("")
  const [longitude, setLongitude] = useState("")
  const [urlMap, setUrlMap] = useState("")

  const getCoordinates = async() => {
    const response = await fetch(url)
    const data = await response.json()
    setLatitude(data["iss_position"]["latitude"])
    setLongitude(data["iss_position"]["longitude"])

    const iss_long = data["iss_position"]["longitude"]
    const iss_lat = data["iss_position"]["latitude"]

    setUrlMap(`https://mapy.cz/zakladni?x=${iss_long}&y=${iss_lat}&z=5`)
  }

  useEffect( () => {
    getCoordinates()
  }, [])

  return (
    <div>
      <h1>API</h1>
      <h2>Zeměpisná šířka</h2>
    </div>
  )
}
```

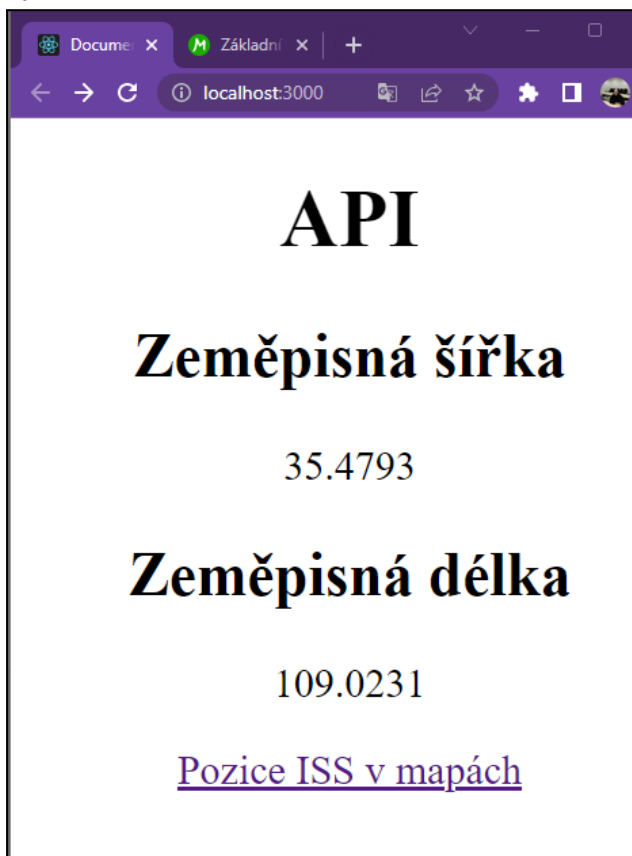
```

    <p>{latitude}</p>
    <h2>Zeměpisná délka</h2>
    <p>{longitude}</p>
    <a href={urlMap} target="_blank">Pozice ISS v mapách</a>
  </div>
)
}

export default App

```

Výsledek v prohlížeči



## 73. React - Multiple returns a k čemu v reactu je

Video: <https://youtu.be/w8dY31UU7-s>

Více return v kódu můžeme použít k tomu, aby se v určité situaci vrátil nějaký kód, ale v jiné situaci jiný kód. Pokud loading bude nastaveno na true, tak se vrátí H2 s textem “Načítání stránky”. Pokud bude loading false, tak se vrátí H2 s textem “Obsah stránky”

```
// Multiple returns

const App = () => {
  const loading = false

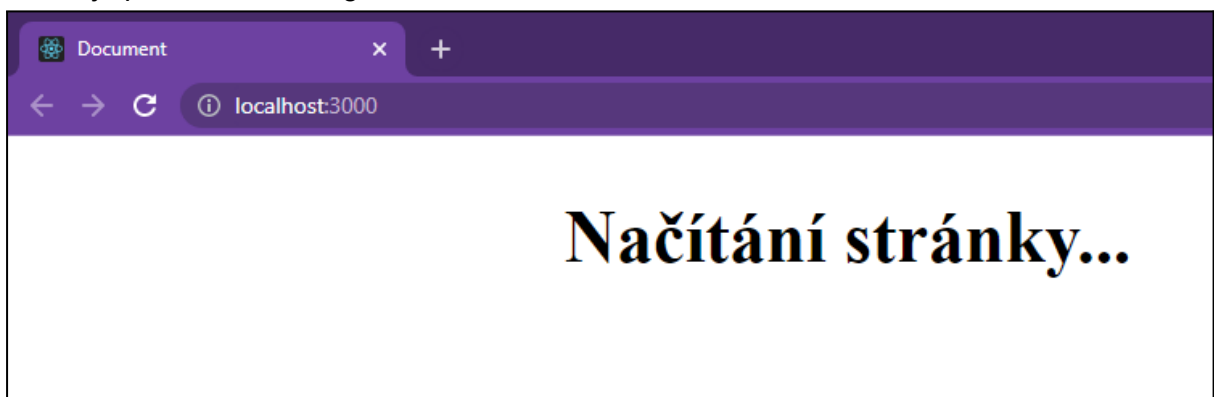
  if (loading) {
    return <h2>Načítání stránky...</h2>
  } else {
    return <h2>Obsah stránky</h2>
  }
}

export default App
```

Pokud je proměnná loading false



Pokud je proměnná loading true



## 74. React - Jak lépe pracovat s API

Video: [https://youtu.be/zT\\_Llt8GJXg](https://youtu.be/zT_Llt8GJXg)

Naimportujeme `useState` a `useEffect` a použijeme stejnou adresu pro API jako v předchozích videích:

International space station API

<http://open-notify.org/Open-Notify-API/ISS-Location-Now/>

data jsou na této adrese: <http://api.open-notify.org/iss-now.json>

Adresu s daty uložíme do proměnné `url`.

Proměnnou `loading` budeme nastavovat pomocí `useState`, aby se v určitý moment mohla proměnná změnit na `false` a komponenta se znovu do stránky vypsala.

V `useEffect` pracujeme s API. Máme zde ukázkou jiného přístupu při práci s API, než tomu bylo v předchozích případech. Používáme `then`. To čeká, než se dokončí předchozí řádek kódu a výsledek pošle do dalšího `then`.

```
// Multiple returns
import { useState, useEffect } from "react"
const url = "http://api.open-notify.org/iss-now.json"

const App = () => {
  const [loading, setLoading] = useState(true)

  useEffect( () => {
    fetch(url)
      .then( (response) => response.json() )
      .then( (data) => data["iss_position"] )
      .then( (position) => {
        console.log(position["latitude"])
        console.log(position["longitude"])
      })
  }, [])

  if (loading){
    return <h2>Načítání stránky...</h2>
  } else {
    return <h2>Obsah stránky</h2>
  }
}

export default App
```

## 75. React - Vypisujeme aktuální data z API do stránky

Video: <https://youtu.be/0s6H3IGDH> 8

Nastavte si loading na false.

Vytvoříme dvě proměnné (latitude a longitude), které budeme nastavovat pomocí useState.

Nastavíme je jako prázdné stringy.

Pomocí setLatitude a setLongitude nastavíme proměnné latitude a longitude na aktuální hodnoty mezinárodní vesmírné stanice.

Vše vypíšeme do stránky v return.

```
// Multiple returns
import { useState, useEffect } from "react"
const url = "http://api.open-notify.org/iss-now.json"

const App = () => {
  const [loading, setLoading] = useState(false)
  const [latitude, setLatitude] = useState("")
  const [longitude, setLongitude] = useState("")

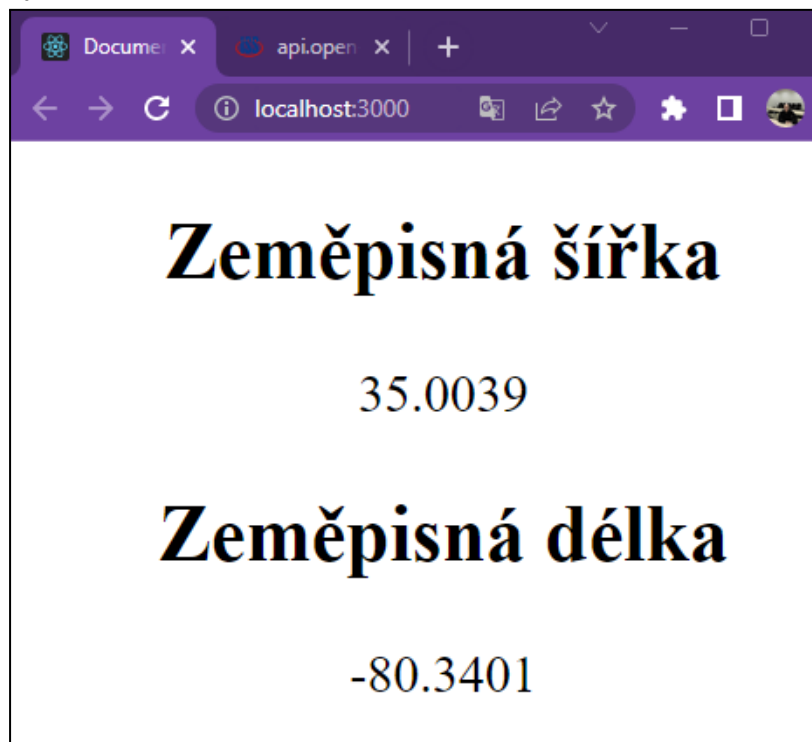
  useEffect( () => {
    fetch(url)
      .then( (response) => response.json() )
      .then( (data) => data["iss_position"] )
      .then( (position) => {
        setLatitude(position["latitude"])
        setLongitude(position["longitude"])
      })
  }, [])

  if (loading){
    return <h2>Načítání stránky...</h2>
  }

  return <>
    <h2>Zeměpisná šířka</h2>
    <p>{latitude}</p>
    <h2>Zeměpisná délka</h2>
    <p>{longitude}</p>
  </>
}

export default App
```

Výsledek v prohlížeči



## 76. React - Jak pracovat s nápisem Načítání stránky

Video: <https://youtu.be/m09Ifv9rQoE>

Nastavte si proměnnou loading na true.

Jakmile se data donáčtou, tak nastavíme loading na false a tím zmizí nápis "Načítání stránky..." z prohlížeče a vypíše se obsah druhého return (zeměpisná šířka a délka).

```
// Multiple returns
import { useState, useEffect } from "react"
const url = "http://api.open-notify.org/iss-now.json"

const App = () => {
  const [loading, setLoading] = useState(true)
  const [latitude, setLatitude] = useState("")
  const [longitude, setLongitude] = useState("")

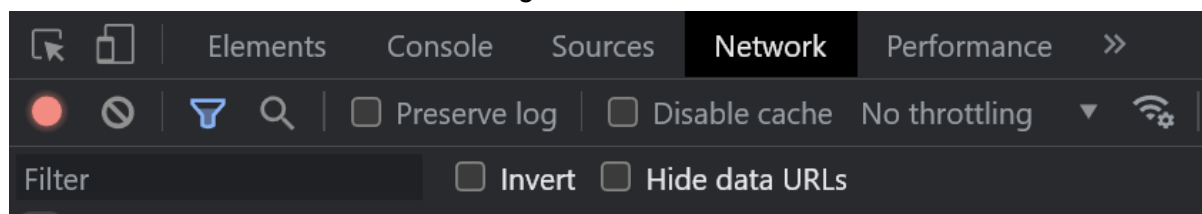
  useEffect( () => {
    fetch(url)
      .then( (response) => response.json() )
      .then( (data) => data["iss_position"] )
      .then( (position) => {
        setLatitude(position["latitude"])
        setLongitude(position["longitude"])
      })
    setLoading(false)
  }, [])

  if (loading){
    return <h2>Načítání stránky...</h2>
  }

  return <>
    <h2>Zeměpisná šířka</h2>
    <p>{latitude}</p>
    <h2>Zeměpisná délka</h2>
    <p>{longitude}</p>
  </>
}

export default App
```

Tip - pokud potřebujete simulovat pomalejší internetové připojení, tak můžete v konzoli v záložce Network rozkliknout No throttling a tam zadat Slow 3G.



## 77. React - Short circuit evaluation (1. část)

Video: <https://youtu.be/8MGb0XPpohU>

### && (pravý Alt + C)

Pokud používáme “a zároveň”, tak pokud je první hodnota true, tak se vždy vrátí druhá hodnota. Pokud je první hodnota false, tak se vrátí první hodnota.

### || (pravý Alt + w)

Pokud používáme “nebo”, tak pokud je první hodnota true, tak vrátí první hodnotu. Pokud je první hodnota false, tak vrátí rovnou druhou hodnotu.

```
// Short circuit evaluation
const App = () => {
  const value = "david"
  const value2 = "harry"

  const result1 = value && value2
  // první je true a druhé jakékoli = se vrátí druhá hodnota
  // první je false a druhé jakékoli = vrátí se první hodnota
  console.log(result1)

  const result2 = value || "tomas"
  // první je true a druhé jakékoli = vrátí první
  // první je false, tak vrátí druhou
  // console.log(result2)

  return <div>

  </div>
}

export default App
```

## 78. React - Short circuit evaluation (2. část)

Video: <https://youtu.be/jnlzkTcVlg0>

Nyní se zaměříme na ukázky dole v return.

Pokud je value2 = 0

- tak se vrátí defaultní text u &&
- tak se vrátí nula u ||

Pokud je value2 = 1

- tak se vrátí 1 u &&
- tak se vrátí Super tajný nadpis u ||

Přemýšlejme nad tím jako nad přepínáním

```
// Short circuit evaluation
const App = () => {
  const value = "david"
  const value2 = 1

  const result1 = value && value2
  // první je true = se vrací druhá hodnota
  // první je false = vrací se první hodnota
  // console.log(result1)

  const result2 = value || "tomas"
  // první je true = vrací první hodnota
  // první je false = tak vrací druhou hodnotu
  // console.log(result2)

  return <div>
    <h1>{value2 || "Defaultní text"}</h1>
    {value2 && <h1>Super tajný nadpis</h1>}
  </div>
}

export default App
```

## 79. React - Ternární operátor (ternary operator)

Video: [https://youtu.be/DrQx\\_jz\\_7nA](https://youtu.be/DrQx_jz_7nA)

Ternární operátor je jiný způsob zápisu podmínky. Pokud je proměnná error true, tak se provede část před dvojtečkou. Pokud bude error false, tak se provede část za dvojtečkou. Můžete zkusit přepisovat proměnnou error z true na false a opačně. Uvidíte, že se výpis na stránce mění.

```
// Ternární operátor

const App = () => {
  const error = true

  return <div>
    {error ? <p>Chyba</p> : <div>
      <h2>Obsah stránky</h2>
      <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.
Distinctio facilis a asperiores ut eaque, sunt reprehenderit expedita
labore recusandae, perspiciatis error aliquid laudantium, itaque quo.
Dolore ut veritatis a ratione?</p>
      </div>}
    </div>
  }

export default App
```

## 80. React - Ukaž a skryj komponentu

Video: <https://youtu.be/AVbm5WkijzY>

App.js

Show pomocí useState nastavíme na false. Když dojde ke kliknutí na tlačítko, tak pomocí setShow nastavíme show na opak toho, co v show zrovna je. Pokud je tam true, tak do show nastavíme false. Pokud je tam false, tak do show nastavíme true. Tím způsobíme, že se bude objevovat a mizet komponenta Book. Pokud je show false, tak se nevrací komponenta Book. Pokud je show true, tak se vrátí komponenta Book.

```
// Ukaž a skryj
import { useState } from "react"
import Book from "../components/Book"

const App = () => {
  const [show, setShow] = useState(false)

  return <div>
    <button onClick={() => setShow(!show)}>Ukaž / skryj</button>
    {show && <Book />}
  </div>
}

export default App
```

Book.js

```
import "../Book.css"

const Book = () => {
  return <div>
    <h1>Harry Potter a Kámen mudrců</h1>
  </div>
}

export default Book
```

Book.css

```
h1 {
  text-decoration: underline;
}
```

Výsledek v prohlížeči

[Ukaž / skryj](#)

# Harry Potter a Kámen mudrců

## 81. React - Projekt - překlikávání filmů - ukázka

Video: <https://youtu.be/OwqiFwGOwRw>

Takto bude vypadat aplikace, kterou nyní společně vytvoříme



## 82. React - Projekt překlíkávání filmů - připravujeme data

Video: [https://youtu.be/xfWE61-n\\_TM](https://youtu.be/xfWE61-n_TM)

data.js

Do data.js si zkopírujete tato data

```
import image1 from "../images/movie1.png"
import image2 from "../images/movie2.png"
import image3 from "../images/movie3.png"
import image4 from "../images/movie4.png"
import image5 from "../images/movie5.png"
import image6 from "../images/movie6.png"

const allMovies = [
  {
    id: 1,
    image: image1,
    title: "Enola Holmesová 2",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
    category: "komedie"
  },
  {
    id: 2,
    image: image2,
    title: "Enola Holmesová",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
    category: "komedie"
  },
  {
    id: 3,
    image: image3,
    title: "Rocknrolla",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
```

```

      description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
      category: "romantický"
    },
    {
      id: 4,
      image: image4,
      title: "Patriot",
      age: "+16",
      tags: "Gangster * Napínavý * Násilí",
      description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
      category: "romantický"
    },

    {
      id: 5,
      image: image5,
      title: "Telefon pana Harrigana",
      age: "+16",
      tags: "Gangster * Napínavý * Násilí",
      description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
      category: "akční"
    },
    {
      id: 6,
      image: image6,
      title: "Stážista",
      age: "+16",
      tags: "Rodinný * Napínavý * Romantický",
      description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
      category: "akční"
    },
  ]

export default allMovies

```

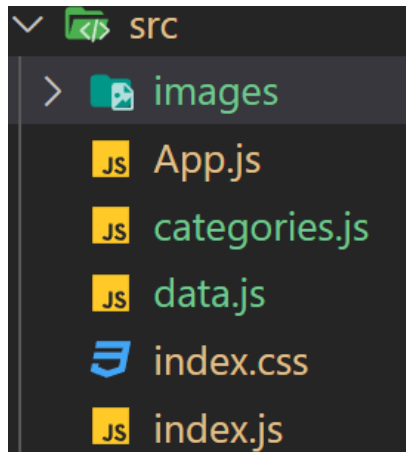
categories.js

Do souboru categories si vytvoříme pole, které bude obsahovat jednotlivé kategorie filmů

```
const categories = ["komedie", "romantický", "akční"]  
  
export default categories
```

složka images

Vytvoříme si novou složku images uvnitř složky src



Do složky images stáhneme všechny tyto obrázky

<https://drive.google.com/drive/folders/1td236jZt7gvk1WxD2YzB9Edot7AxMk2P?usp=sharing>

## 83. React - Projekt překlikávání filmů - filtrování filmů

Video: <https://youtu.be/F5ogUj8a9Ms>

### App.js

Nejprve si načteme do App.js obsah souboru data a categories.

Poté vytvoříme proměnnou typeOfMovie a nastavíme ji na text “komedie”.

Poté vezeme všechny filmy a vyfiltrujeme z celého seznamu pouze ty filmy, u kterých se jejich kategorie rovná obsahu proměnné typeOfMovie (tedy v tuto chvíli komedie). Proměnná vysledneFilmy tak obsahuje pole filmů, které mají jako kategorii uvedenou komedie.

```
import { useState } from "react"
import allMovies from "../data"
import categories from "../categories"

const App = () => {
  const [typeOfMovie, setTypeOfMovie] = useState("komedie")

  const vysledneFilmy = allMovies.filter( (oneMovie) => {
    return oneMovie["category"] === typeOfMovie
  })

  console.log(vysledneFilmy)

  return <div>

  </div>
}

export default App
```

## 84. React - Projekt překlikávání filmů - vypisujeme filmy do stránky

Video: <https://youtu.be/SwFjRNZ230k>

App.js

Vezmeme obsah proměnné `vysledneFilmy` a vypíšeme je do stránky. K tomu použijeme funkci `map` a také tzv. destructuring na jednotlivé prvky objektu (jednoho filmu)

```
import { useState } from "react"
import allMovies from "../data"
import categories from "../categories"

const App = () => {
  const [typeOfMovie, setTypeOfMovie] = useState("komedie")

  const vysledneFilmy = allMovies.filter( (oneMovie) => {
    return oneMovie["category"] === typeOfMovie
  })

  return <div>
    <div>
      {
        vysledneFilmy.map( (oneMovie) => {
          const {id, image, title, age, tags, description} = oneMovie

          return <div>
            <img src={image} alt="" />
            <h2>{title}</h2>
            <p>{age}</p>
            <p>{tags}</p>
            <p>{description}</p>
          </div>
        })
      }
    </div>
  </div>
}

export default App
```

## 85. React - Projekt překlíkávání filmů - tlačítka do stránky

Video: <https://youtu.be/HAIFHPSBfDk>

App.js

Pomocí map vypíšeme z proměnné categories všechny názvy kategorií a doplníme je jako text tlačítek.

Poté na tlačítka přidáme událost kliknutí a pokud se na dané tlačítko klikne, tak nastavíme proměnnou typeOfMovie na hodnotu tlačítka, na které bylo kliknuto. Pokud se klikne na tlačítko s textem “romantický”, tak se typeOfMovie nastaví na “romantický” a useState znovu vypíše komponentu do stránky, takže ve vysledneFilmy budou romantické filmy.

Také doplníme dva unikátní identifikátory.

```
const App = () => {
  const [typeOfMovie, setTypeOfMovie] = useState("komedie")

  const vysledneFilmy = allMovies.filter( (oneMovie) => {
    return oneMovie["category"] === typeOfMovie
  })

  return <div>
    <div>
      {
        categories.map( (oneCategory, index) => {
          return <button key={index} onClick={() =>
setTypeOfMovie(oneCategory)}>
            {oneCategory}
          </button>
        })
      }
    </div>
    <div>
      {
        vysledneFilmy.map( (oneMovie) => {
          const {id, image, title, age, tags, description} = oneMovie

          return <div key={id}>
            <img src={image} alt="" />
            <h2>{title}</h2>
            <p>{age}</p>
            <p>{tags}</p>
            <p>{description}</p>
          </div>
        })
      }
    </div>
  </div>
```

```
</div>  
}  
  
export default App
```

## 86. React - Projekt překlíkávání filmů - stylování aplikace

Video: <https://youtu.be/6JRYtFU5gRE>

App.js

Do App.js přidáme čtyři nové classy, abychom přes ně mohli prvky stylovat

```
return <div >
  <div className="all-buttons">
    {
      categories.map((oneCategory) => {
        return <button className="one-button" onClick={() => {
setTypeOfMovie(oneCategory) }}>{oneCategory}</button>
      })
    }
  </div>
  <div className="all-movies">
    {
      vysledneFilmy.map((oneMovie) => {

        const {id, image, title, age, tags, description} = oneMovie

        return <div className="one-movie" key={id}>
          <img src={image} alt="" />
          <h2>{title}</h2>
          <p>{age}</p>
          <p>{tags}</p>
          <p>{description}</p>
        </div>
      })
    }
  </div>
</div>
```

## Index.css

```
body {
  background-color: #141414;
}

.all-buttons {
  display: flex;
  flex-direction: row;
  justify-content: center;
  margin-top: 50px;
  margin-bottom: 50px;
}

.one-button {
  color: white;
  background-color: #e50914;
  padding: 5px 10px;
  font-size: 18px;
  cursor: pointer;
  margin-left: 5px;
  margin-right: 5px;
}

.all-movies {
  width: 90%;
  display: flex;
  flex-direction: row;
  justify-content: center;
  flex-wrap: wrap;
  margin: 0px auto;
}

.one-movie {
  width: 300px;
  box-shadow: 0px 0px 20px black;
  color: #e5e5e5;
  margin: 10px 5px;
  display: flex;
  flex-direction: column;
  transition: all 0.5s linear;
}

.one-movie:hover {
```

```

    transform: scale(1.04);
}

.one-movie img {
    height: 170px;
}

.one-movie h2,
.one-movie p {
    padding-left: 10px;
    margin-top: 10px;
    margin-bottom: 10px;
}

```

Výsledek v prohlížeči



## 87. React - Projekt překlíkávání filmů - výhoda komponent a rozšiřování

Video: <https://youtu.be/Vo0EsWKcHWU>

data.js

Do data.js můžeme přidat další dva filmy, které mají kategorii "horor"

```
{
  id: 7,
  image: image7,
  title: "Život",
  age: "+13",
  tags: "Vesmír * Napínavý * Nebezpečí",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  category: "horor"
},
{
  id: 8,
  image: image8,
  title: "Kód za miliardu dolarů",
  age: "+13",
  tags: "IT * Napínavý",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  category: "horor"
},
```

categories.js

Do souboru přidáme další kategorii.

```
const categories = ["komedie", "romantický", "akční", "horor"]

export default categories
```

Výsledek v prohlížeči

komedie

romantický

akční

horor



**Život**

+13

Vesmír \* Napínavý \* Nebezpečí

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium amet nemo minus quo architecto quas



**Kód za miliardu dolarů**

+13

IT \* Napínavý

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Autem accusantium amet nemo minus quo architecto quas

## 88. React - Projekt FAQ otázky a odpovědi - ukázka

Video: <https://youtu.be/PNyuaVDyj2o>

Takto bude vypadat naše aplikace.



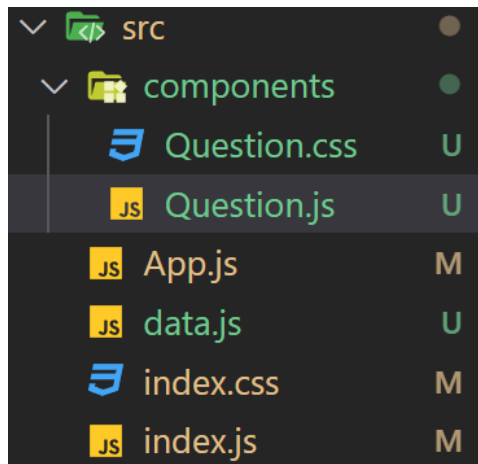
Takto bude aplikace vypadat, pokud klikneme na Odpověď



## 89. React - Projekt FAQ otázky a odpovědi - data a komponenta

Video: <https://youtu.be/-iVacPNtZwM>

Obsah složky src bude vypadat takto:



data.js

Tato data si zkopírujeme do data.js. Pozor, ať zkopírujete i export na konci dat.

```
const questions = [
  {
    id: 1,
    title: "Jaké typy filmů najdu na Netflixu?",
    info: "Lorem ipsum dolor sit amet, consectetur adipisicing elit. Illo vitae dolores doloremque facilis a earum error dignissimos modi possimus harum."
  },
  {
    id: 2,
    title: "Jak se na Netflix přihlásím?",
    info: "Consectetur adipisicing elit. Illo vitae dolores doloremque facilis a earum error dignissimos modi possimus harum, iste, accusantium numquam magni excepturi maiores rem molestiae in maxime."
  },
  {
    id: 3,
    title: "Kolik stojí měsíční členství na Netflixu?",
    info: "Sit amet, consectetur adipisicing elit. Illo vitae dolores doloremque facilis a earum error dignissimos modi possimus harum."
  },
  {
    id: 4,
    title: "Jak často Netflix přidává nové filmy?",
    info: "Iste, accusantium numquam magni excepturi maiores rem molestiae in maxime."
  }
]

export default questions
```

### App.js

Naimportujeme si data a také komponentu Question (tu vytváříme dále). 5x komponentu Question použijeme v kódu.

```
import questions from "../data"
import Question from "../components/Question"

const App = () => {
  return <div>
    <Question />
    <Question />
    <Question />
    <Question />
    <Question />
  </div>
}

export default App
```

### Question.js

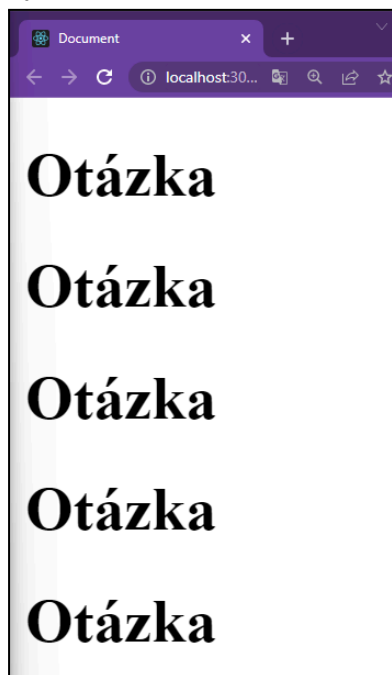
Takto vypadá komponenta Question. Naimportované styly a v return jen h1 s libovolným textem.

```
import "../Question.css"

const Question = () => {
  return (
    <h1>Otázka</h1>
  )
}

export default Question
```

Výsledek na stránce



## 90. React - Projekt FAQ otázky a odpovědi - tvoříme komponentu

Video: <https://youtu.be/17ZNT640MvA>

Question.js

Vytvoříme komponentu Question, kde bude otázka (title) a odpověď (info).

Dále vytvoříme proměnnou show, která nám bude sloužit jako přepínač. Uložíme do ní false a po každém kliknutí na tlačítko (button) se obsah show přemění na opačnou hodnotu.

Takže po prvním kliknutí na true. Po druhém kliknutí na false. Po třetím kliknutí na true a tak stále dokola.

Poslední řádek v return zabezpečuje, že podle toho, jaké hodnoty nabývá show, tak se <p> zobrazuje nebo ne.

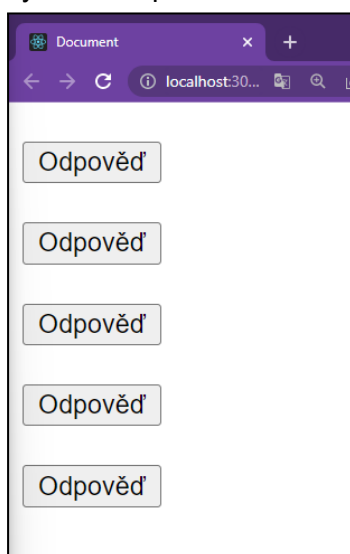
```
import "../Question.css"
import { useState } from "react"

const Question = ({title, info}) => {
  const [show, setShow] = useState(false)

  return <div>
    <section>
      <h2>{title}</h2>
      <button onClick={ () => setShow(!show)}>Odpověď</button>
    </section>
    {show && <p>{info}</p>}
  </div>
}

export default Question
```

Výsledek v prohlížeči



## 91. React - Projekt FAQ otázky a odpovědi - map a rest parametr

Video: [https://youtu.be/D3g\\_p1cCZyl](https://youtu.be/D3g_p1cCZyl)

App.js

Pomocí funkce map začneme procházet naše data a promítneme je do komponenty Question. Zadáme id jako unikátní identifikátor a zbytek dat (title a info) pošleme přes tzv. rest parameter, který vezme vždy zbytek a pošle ho do Question.

```
import questions from "../data"
import Question from "../components/Question"

const App = () => {
  return <div>
    {
      questions.map((oneQuestion) => {
        return <Question key={oneQuestion.id} {...oneQuestion}/>
      })
    }
  </div>
}

export default App
```

Výsledek na stránce - po kliknutí na Odpověď se zobrazí odstavec <p>



## 92. React - Projekt FAQ otázky a odpovědi - stylování aplikace

Video: <https://youtu.be/tGnrTw3ZY04>

index.css

```
body {
  background-color: #141414;
  margin: 0;
}

#result {
  height: 100vh;
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items: center;
}

.all-questions {
  color: #e5e5e5;
}
```

Question.css

```
.one-question {
  box-shadow: 0px 0px 10px #e50914;
  width: 800px;
  margin-bottom: 20px;
  padding: 10px;
}

.one-question section {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
}

.one-question h2 {
  font-weight: 400;
  letter-spacing: 1px;
  margin-left: 50px;
}

.one-question button {
```

```

height: 40px;
margin-right: 50px;
color: #e5e5e5;
background-color: #e50914;
padding: 5px 10px;
font-size: 18px;
cursor: pointer;
}

.one-question p {
margin: 0px 50px 10px;
}

```

Výsledek v prohlížeči

|                                           |         |
|-------------------------------------------|---------|
| Jaké typy filmů najdu na Netflixu?        | Odpověď |
| Jak se na Netflix přihlásím?              | Odpověď |
| Kolik stojí měsíční členství na Netflixu? | Odpověď |
| Jak často Netflix přidává nové filmy?     | Odpověď |

Po kliknutí na Odpověď

|                                                                                                                                                                                                        |         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <p><b>Jaké typy filmů najdu na Netflixu?</b></p> <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Illo vitae dolores doloremque facilis a earum error dignissimos modi possimus harum.</p> | Odpověď |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|

### 93. React - Projekt FAQ otázky a odpovědi - přidání dalších otázek a odpovědí

Video: <https://youtu.be/iIRj6H Lw7o>

data.js

Na konec data.js si můžete přidat další otázku a odpověď.

```
{
  id: 5,
  title: "V kolika zařízeních mohu Netflix používat?",
  info: "Iste, accusantium numquam magni excepturi maiores rem molestiae in maxime."
},
```

Výsledek v prohlížeči - vidíme, že se okamžitě zobrazí na svém místě a nastylovaná. Rozšiřování je díky Reactu velice jednoduché



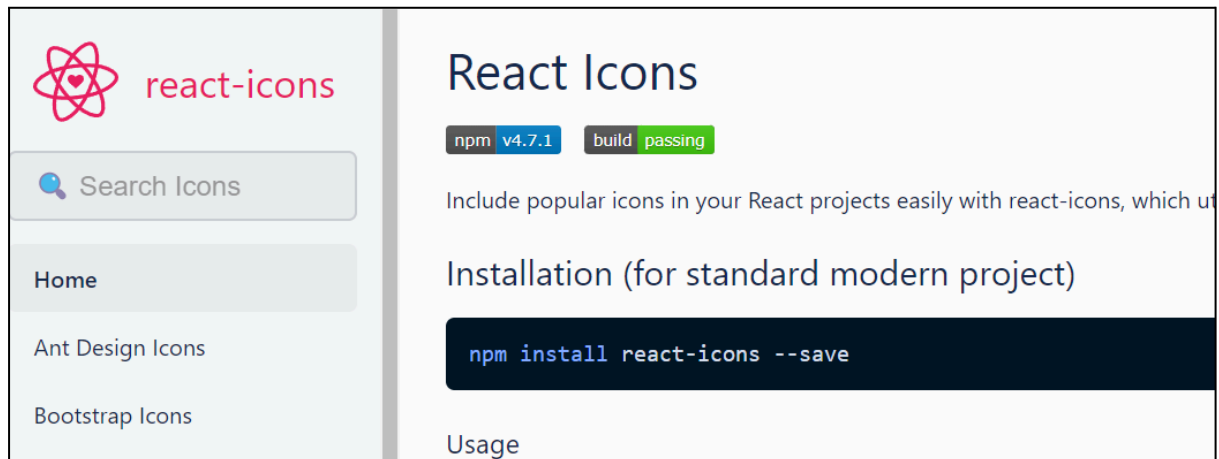
## 94. React - Jak pracovat s ikonkami

Video: <https://youtu.be/QiPUjbyUfxY>

Nainstalujeme si tuto knihovnu ikon:

<https://react-icons.github.io/react-icons/>

Stačí zadat do terminálu příkaz, který vidíme na obrázku



App.js

Poté už jen kopírujeme kódy z různých skupin ikon, které vidíme na stránce vlevo a zadáváme do nich názvy konkrétních ikon, které chceme používat.

```
import { FaAvianex } from "react-icons/fa";
import { DiApple } from "react-icons/di";

function App() {

  return <div>
    <h1>Text</h1>
    <FaAvianex className="my-icon"/>
    <DiApple className="my-icon" />
  </div>

}

export default App;
```

## 95. React - Projekt slider - ukázka

Video: <https://youtu.be/PNu1WMql92U>

Takto bude vypadat náš slider



## 96. React - Projekt slider - připravujeme data a obrázky

Video: <https://youtu.be/ow8TCav8i8U>

Instalace ikonek:

<https://react-icons.github.io/react-icons/>

Založíme novou složku images (uvnitř složky src) a do ní stáhneme tyto obrázky:

<https://drive.google.com/drive/folders/1td236jZt7gvk1WxD2YzB9Edot7AxMk2P?usp=sharing>

data.js

Ve složce src vytvoříme nový soubor data.js a zkopírujeme do něj předpřipravená data.

Včetně importu obrázků. Nezapomeňte na konci zkopírovat také řádek s exportem.

```
import image1 from "../images/movie1.png"
import image2 from "../images/movie2.png"
import image3 from "../images/movie3.png"
import image4 from "../images/movie4.png"

const allMovies = [
  {
    id: 1,
    image: image1,
    title: "Enola Holmesová 2",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Nulla pulvinar eleifend sem. Mauris suscipit, ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis velit mauris vel metus. Nam libero tempore, cum soluta",
    category: "komedie"
  },
  {
    id: 2,
    image: image2,
    title: "Enola Holmesová",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Etiam egestas wisi a erat. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Integer imperdiet lectus quis justo. Vivamus luctus egestas leo",
    category: "komedie"
  },
  {
    id: 3,
    image: image3,
```

```

    title: "Rocknrolla",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
    description: "Nulla pulvinar eleifend sem. Mauris suscipit,
ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis
velit mauris vel metus. Nam libero tempore, cum soluta",
    category: "romantický"
  },
  {
    id: 4,
    image: image4,
    title: "Patriot",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
    description: "Duis ante orci, molestie vitae vehicula
venenatis, tincidunt ac pede. Nulla quis diam. Nulla pulvinar eleifend
sem. Maecenas lorem. Fusce suscipit libero eget.",
    category: "romantický"
  },
]

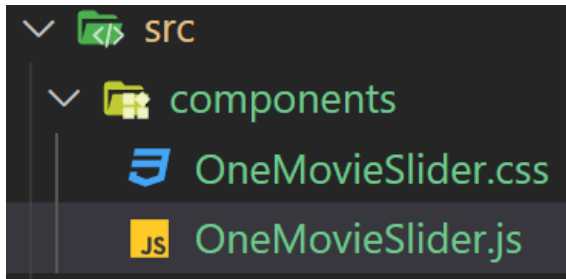
export default allMovies

```

## 97. React - Projekt slider - zobrazujeme filmy

Video: <https://youtu.be/jxYwP7O24WE>

Ve složce src si založíme novou složku components. Do ní vytvoříme dva soubory OneMovieSlider.js a OneMovieSlider.css



OneMovieSlider.js

Naimportujeme vše potřebné - CSS, data, useState a také dvě ikonky, které představují šipky doleva a doprava.

Pomocí useState nastavíme proměnnou index na 0 (0. film z pole). Z dat si vytáhneme pouze to, co budeme potřebovat - image, title, description. Umístíme do return jak potřebujeme. Také doplníme dvě tlačítka jejichž obsahem nebude text, ale naimportované ikonky.

```
import "../OneMovieSlider.css"
import data from "../data"
import { useState } from "react"
import { FaArrowAltCircleLeft, FaArrowAltCircleRight } from
"react-icons/fa";

const OneMovieSlider = () => {
  const [index, setIndex] = useState(0)
  const {image, title, description} = data[index]

  return <div>
    <img src={image} alt="" />
    <h2>{title}</h2>
    <p>{description}</p>
    <button>
      <FaArrowAltCircleLeft />
    </button>
    <button>
      <FaArrowAltCircleRight />
    </button>
  </div>
}
```

```
export default OneMovieSlider
```

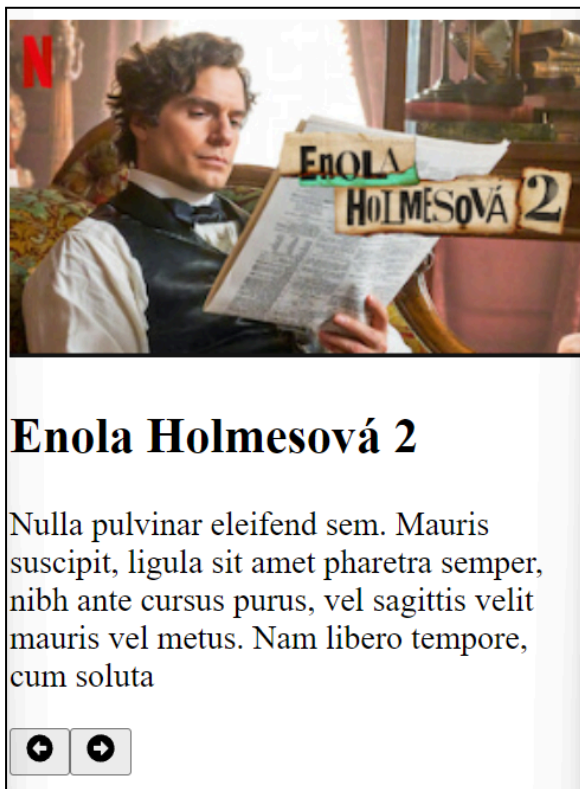
App.js

Do App.js naimportujeme komponentu OneMovieSlider a necháme ji vypsat do stránky.

```
import OneMovieSlider from "../components/OneMovieSlider"
```

```
const App = () => {  
  
  return <div>  
    <OneMovieSlider />  
  </div>  
}  
  
export default App
```

Výsledek v prohlížeči



## 98. React - Projekt slider - překlíkáváme filmy

Video: <https://youtu.be/Nkn3N0NyhjY>

OneMovieSlider.js

Vytvoříme dvě funkce. Funkce nextMovie zvedá index o + 1 a tím se posouváme na další filmy. Funkce previousMovie snižuje index o - 1 a tím se posouváme na předcházející film. Obě funkce jsou navázané na tlačítka (resp. ikonky).

```
const OneMovieSlider = () => {
  const [index, setIndex] = useState(0)
  const {image, title, description} = data[index]

  const nextMovie = () => {
    setIndex( (index) => {
      const newIndex = index + 1
      return newIndex
    })
  }

  const previousMovie = () => {
    setIndex( (index) => {
      const newIndex = index - 1
      return newIndex
    })
  }

  return <div>
    <img src={image} alt="" />
    <h2>{title}</h2>
    <p>{description}</p>
    <button onClick={previousMovie}>
      <FaArrowAltCircleLeft />
    </button>
    <button onClick={nextMovie}>
      <FaArrowAltCircleRight />
    </button>

  </div>
}

export default OneMovieSlider
```

Výsledek v prohlížeči - stále stejný jako v předchozím videu, ale již nám funguje tlačítka.



## Enola Holmesová 2

Nulla pulvinar eleifend sem. Mauris  
suscipit, ligula sit amet pharetra semper,  
nibh ante cursus purus, vel sagittis velit  
mauris vel metus. Nam libero tempore,  
cum soluta



## 99. React - Projekt slider - filmy se mění stále dokola

Video: <https://youtu.be/qwP8Rbfzjkg>

OneMovieSlider.js

Než index nastavíme na hodnotu newIndex, tak ho ještě zkontrolujeme pomocí funkce checkMovieNumber. Pokud je hodnota newIndex menší než 0, tak vrátíme poslední film. Pokud je naopak hodnota newIndex větší než poslední index filmu, tak vrátíme první film. Pokud je newIndex v pořádku, tak ho jen vrátíme.

```
const OneMovieSlider = () => {
  const [index, setIndex] = useState(0)
  const {image, title, description} = data[index]

  const checkMovieNumber = (movieIndex) => {
    if (movieIndex < 0) {
      return data.length - 1
    } else if (movieIndex > data.length - 1) {
      return 0
    } else {
      return movieIndex
    }
  }

  const nextMovie = () => {
    setIndex( (index) => {
      const newIndex = index + 1
      return checkMovieNumber(newIndex)
    })
  }

  const previousMovie = () => {
    setIndex( (index) => {
      const newIndex = index - 1
      return checkMovieNumber(newIndex)
    })
  }
}
```

## 100. React - Projekt slider - stylování aplikace

Video: <https://youtu.be/BPe9p-J-rkw>

Nyní si aplikaci nastylujeme, aby lépe vypadala.

Index.css

```
body {  
  background-color: #141414;  
  margin: 0px;  
}  
  
.slider {  
  height: 100vh;  
  display: flex;  
  flex-direction: row;  
  justify-content: center;  
  align-items: center;  
}
```

OneMovieSlider.css

```
.one-movie {  
  width: 289px;  
  text-align: center;  
  color: #e5e5e5;  
  box-shadow: 0px 0px 15px #e50914;  
  padding: 10px;  
}  
  
.one-movie img {  
  width: 283px;  
  height: 160px;  
}  
  
.one-movie button {  
  background-color: transparent;  
  border: none;  
  font-size: 25px;  
  color: #e50914;  
  cursor: pointer;  
}
```

Výsledek v prohlížeči



## Enola Holmesová 2

Nulla pulvinar eleifend sem. Mauris suscipit, ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis velit mauris vel metus. Nam libero tempore, cum soluta



## 101. React - Projekt slider - rozšiřování slideru o další filmy

Video: <https://youtu.be/nzcErcVQtsc>

Dva další filmy do data.js

```
{
  id: 5,
  image: image5,
  title: "Telefon pana Harrigana",
  age: "+16",
  tags: "Gangster * Napínavý * Násilí",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  category: "akční"
},
{
  id: 6,
  image: image6,
  title: "Stážista",
  age: "+16",
  tags: "Rodinný * Napínavý * Romantický",
  description: "Lorem ipsum dolor sit, amet consectetur
adipisicing elit. Autem accusantium amet nemo minus quo architecto
quas",
  category: "akční"
},
```

Musíme naimportovat další dva obrázky

```
import image1 from "./images/movie1.png"
import image2 from "./images/movie2.png"
import image3 from "./images/movie3.png"
import image4 from "./images/movie4.png"
import image5 from "./images/movie5.png"
import image6 from "./images/movie6.png"
```

Výsledek v prohlížeči - přidali jsme do slideru dva další filmy



## Stážista

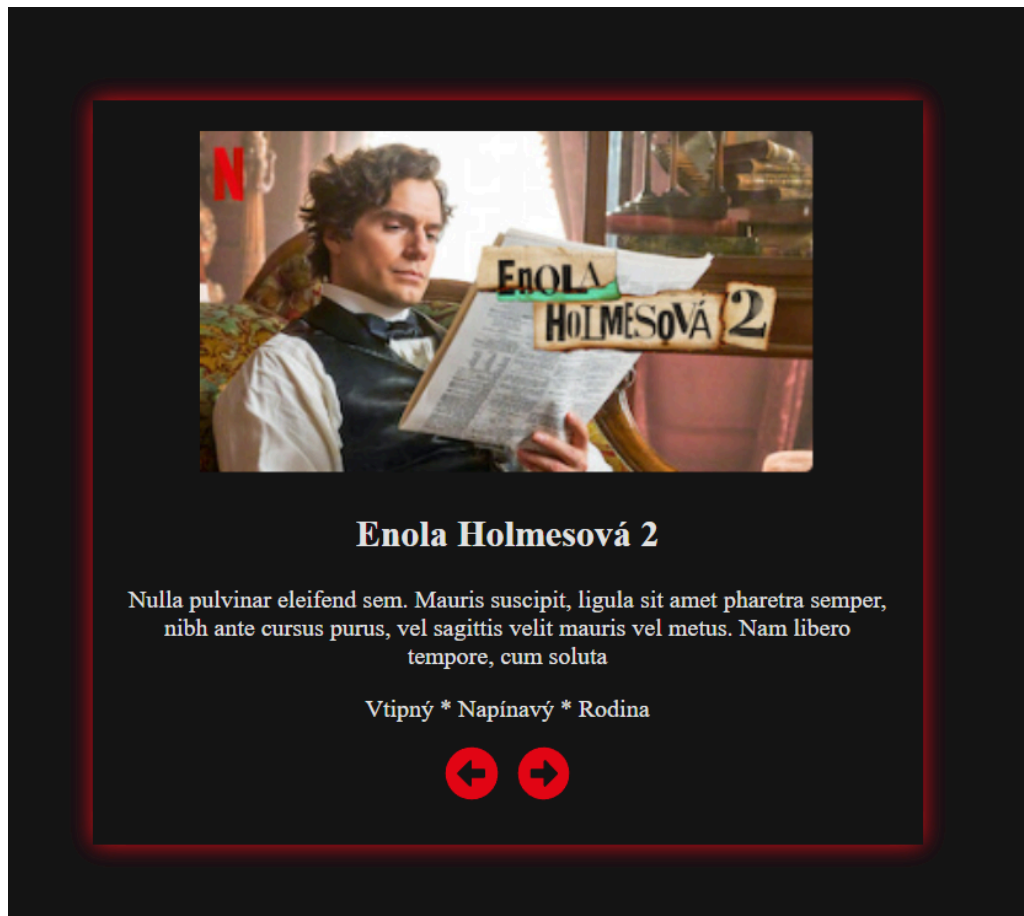
Lorem ipsum dolor sit, amet consectetur  
adipiscing elit. Autem accusantium amet  
nemo minus quo architecto quas

◀ ▶

## 102. React - Projekt pokročilý slider - ukázka

Video: <https://youtu.be/nzcErcVQtsc>

Takto bude vypadat naše aplikace



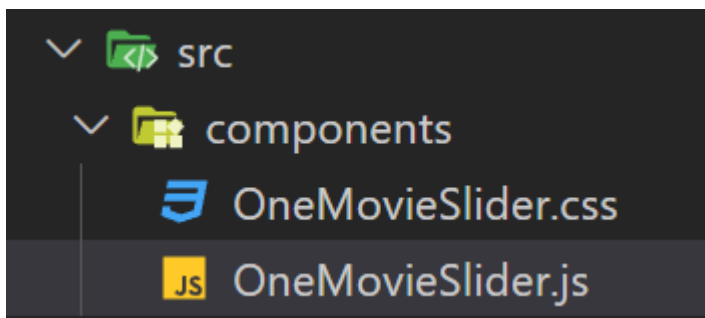
## 103. React - Projekt pokročilý slider - nastavujeme projekt

Video: <https://youtu.be/9W4JwBpy1sc>

Instalace ikonek:

<https://react-icons.github.io/react-icons/>

Ve složce src založíme složku components a v ní dva soubory - OneMovieSlider.js a OneMovieSlider.css.



OneMovieSlider.js

```
import "../OneMovieSlider.css"
import { FaArrowAltCircleLeft, FaArrowAltCircleRight } from
"react-icons/fa";

const OneMovieSlider = () => {
  return <div>
    <h2>Test</h2>
  </div>
}

export default OneMovieSlider
```

App.js

```
import OneMovieSlider from "../components/OneMovieSlider"

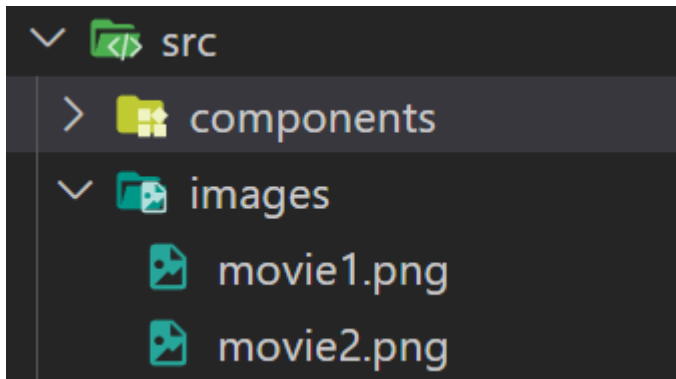
const App = () => {
  return <div>
    <OneMovieSlider />
  </div>
}

export default App
```

Ve složce src založíme složku images a stáhneme do ní obrázky z tohoto odkazu:

<https://drive.google.com/drive/folders/1td236jZt7gvyk1WxD2YzB9Edot7AxMk2P?usp=sharing>

Výsledek vypadá takto:



Ve složce src založíme soubor data.js a zkopírujeme do něj tento obsah.

```
const allMovies = [
  {
    id: 1,
    image: image1,
    title: "Enola Holmesová 2",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Nulla pulvinar eleifend sem. Mauris suscipit,
ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis
velit mauris vel metus. Nam libero tempore, cum soluta",
    category: "komedie"
  },
  {
    id: 2,
    image: image2,
    title: "Enola Holmesová",
    age: "+13",
    tags: "Vtipný * Napínavý * Rodina",
    description: "Etiam egestas wisi a erat. Sed vel lectus. Donec
odio tempus molestie, porttitor ut, iaculis quis, sem. Integer
imperdiet lectus quis justo. Vivamus luctus egestas leo",
    category: "komedie"
  },
  {
    id: 3,
    image: image3,
    title: "Rocknrolla",
    age: "+16",
    tags: "Gangster * Napínavý * Násilí",
```

```

        description: "Nulla pulvinar eleifend sem. Mauris suscipit,
        ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis
        velit mauris vel metus. Nam libero tempore, cum soluta",
        category: "romantický"
    },
    {
        id: 4,
        image: image4,
        title: "Patriot",
        age: "+16",
        tags: "Gangster * Napínavý * Násilí",
        description: "Duis ante orci, molestie vitae vehicula
        venenatis, tincidunt ac pede. Nulla quis diam. Nulla pulvinar eleifend
        sem. Maecenas lorem. Fusce suscipit libero eget.",
        category: "romantický"
    },
]

export default allMovies

```

Do data.js doplníme ještě importy obrázků.

```

import image1 from "../images/movie1.png"
import image2 from "../images/movie2.png"
import image3 from "../images/movie3.png"
import image4 from "../images/movie4.png"

```

## 104. React - Projekt pokročilý slider - vypisujeme filmy do stránky

Video: <https://youtu.be/ZHWYude6xwQ>

OneMovieSlider.js

Naimportujeme useState a useEffect a také data.

Nastavíme proměnnou index na 0 a protože ji budeme muset měnit, tak to uděláme pomocí useState.

Poté do stránky vypíšeme všechny filmy.

```
import './OneMovieSlider.css'
import { FaArrowAltCircleLeft, FaArrowAltCircleRight } from
'react-icons/fa';
import { useState, useEffect } from 'react'
import data from '../data'

const OneMovieSlider = () => {
  const [index, setIndex] = useState(0)

  return <section className="all-movies">
    <div className="all-movies-content">
      {data.map( (oneMovie) => {
        const {id, image, title, age, tags, description } =
oneMovie

        return <article key={id}>
          <img src={image} alt="" />
          <h2>{title}</h2>
          <p>{description}</p>
          <p>{tags}</p>
          <p>{age}</p>
        </article>
      })}
    </div>
    <button>
      <FaArrowAltCircleLeft />
    </button>
    <button>
      <FaArrowAltCircleRight />
    </button>
  </section>
}

export default OneMovieSlider
```

Výsledek v prohlížeči



## Enola Holmesová 2

Nulla pulvinar eleifend sem. Mauris suscipit, ligula sit amet pharetra semper, nibh ante cursus purus, vel sagittis velit mauris vel metus. Nam libero tempore, cum soluta

Vtipný \* Napínavý \* Rodina

+13



## Enola Holmesová

Etiam egestas wisi a erat. Sed vel lectus. Donec odio tempus molestie, porttitor ut, iaculis quis, sem. Integer imperdiet lectus quis justo. Vivamus luctus egestas leo

Vtipný \* Napínavý \* Rodina

+13

## 105. React - Projekt pokročilý slider - stylování důležité pro funkčnost

Video: <https://youtu.be/v-8oc3TKviw>

OneMovieSlider.css

```
.all-movies-content {
  display: flex;
  flex-direction: row;
  border: 2px solid red;
  width: 500px;
  /* overflow: hidden; */
  height: 400px;
  text-align: center;
  position: relative;
}

.all-movies-content article {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  /* opacity: 0; */
}

article.active-slide {
  opacity: 1;
  transform: translateX(0);
}

article.last-slide {
  transform: translateX(-100%);
}

article.next-slide {
  transform: translateX(100%)
}
```

App.js

Přidáme classu slider

```
import OneMovieSlider from "../components/OneMovieSlider"

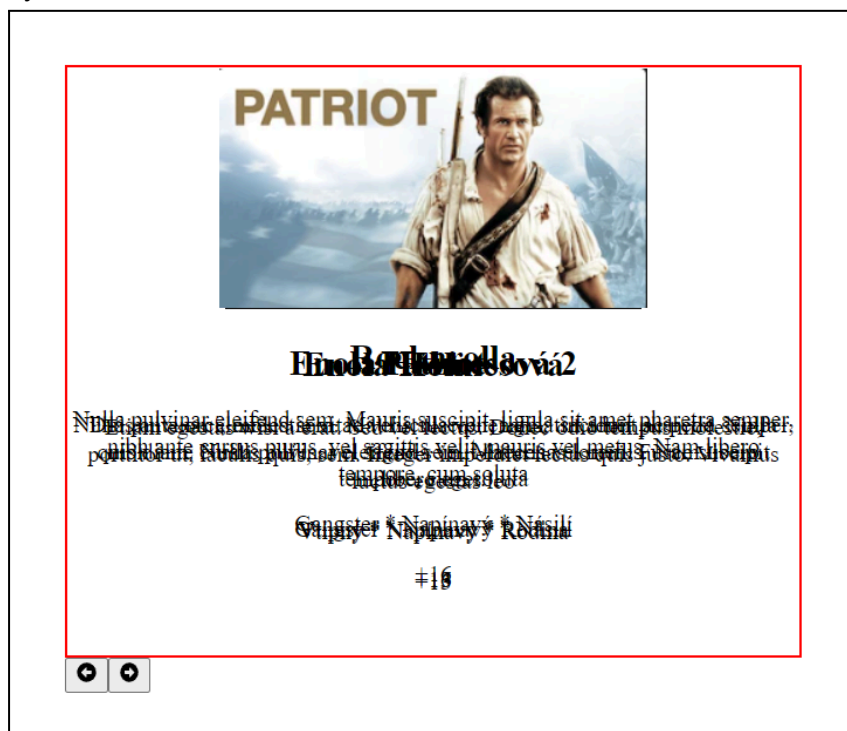
const App = () => {
  return <div className="slider">
    <OneMovieSlider />
  </div>
}

export default App
```

index.css

```
.slider {
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

Výsledek na stránce



## 106. React - Projekt pokročilý slider - přidělování class

Video: [https://youtu.be/0NJ\\_-vQIBSA](https://youtu.be/0NJ_-vQIBSA)

Map dokáže generovat indexy prvků v poli. My si je uložíme do oneMovieIndex. Nyní začneme přiřazovat classy. Všechny filmy nejdříve dostanou next-slide. Otázkou je, jestli se splní některá z dalších podmínek a tato classa bude nahrazena jinou.

```
return <section className="all-movies">
  <div className="all-movies-content">
    {data.map( (oneMovie, oneMovieIndex) => {
      const {id, image, title, age, tags, description} =
oneMovie

      let mainClass = "next-slide"

      if (oneMovieIndex === index){
        mainClass = "active-slide"
      }

      if (oneMovieIndex === index - 1 || (index === 0 &&
oneMovieIndex === data.length - 1)){
        mainClass = "last-slide"
      }

      return <article key={id} className={mainClass}>
        <img src={image} alt="" />
        <h2>{title}</h2>
        <p>{description}</p>
        <p>{tags}</p>
        <p>{age}</p>
      </article>
    })}
  </div>
  <button>
    <FaArrowAltCircleLeft />
  </button>
  <button>
    <FaArrowAltCircleRight />
  </button>
</section>
```

## 107. React - Projekt pokročilý slider - zprovoznění tlačítek a plynulý posun

Video: <https://youtu.be/Ybsg75H7p7M>

OneMovieSlider.js

Zprovozníme tlačítka, aby posouvala filmy doleva a doprava. Zatím to nebude fungovat na krajní hodnoty, protože se dostaneme k hodnotám -1 a 4 a takové indexy neodpovídají žádným filmům.

```
<button onClick={ () => setIndex(index - 1)}>
  <FaArrowAltCircleLeft />
</button>
<button onClick={ () => setIndex(index + 1)}>
  <FaArrowAltCircleRight />
</button>
```

OneMovieSlider.css

Aby se filmy ve slideru pohybovaly plynule, tak přidáme vlastnost transition (přechod). Jakákoliv změna se bude animovat po dobu 0.5 vteřiny lineárním (rovnoměrným) pohybem.

```
.all-movies-content article {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  /* opacity: 0; */
  transition: all 0.5s linear;
}
```

## 108. React - Projekt pokročilý slider - filmy se točí dokola

Video: [https://youtu.be/\\_lQhj5mq-mE](https://youtu.be/_lQhj5mq-mE)

OneMovieSlider.js

Nastavíme, že filmy se mají točit stále dokola. Takže po 3. filmu, bude následovat 0. film.

Před 0. filmem bude naopak 3. film. U nás je 3. film ten poslední. Všimněte si, že v kódu nikde nepíšeme 3 natvrdo, ale zapisujeme ji obecně jako délka pole data mínus jedna.

```
const OneMovieSlider = () => {  
  const [index, setIndex] = useState(0)  
  
  // Filmy se ve slideru točí stále dokola  
  useEffect( () => {  
    if (index < 0){  
      setIndex(data.length - 1)  
    } else if (index > data.length - 1){  
      setIndex(0)  
    }  
  }, [index])  
}
```

## 109. React - Projekt pokročilý slider - automatické posouvání filmů

Video: <https://youtu.be/XD6StQ9yGjs>

OneMovieSlider.js

Nastavíme automatické posouvání pomocí setInterval. Ten udává, jaký kód se má po 3 vteřinách opakovat.

```
// Filmy se ve slideru točí stále dokola
useEffect( () => {
  if (index < 0){
    setIndex(data.length - 1)
  } else if (index > data.length - 1){
    setIndex(0)
  }
}, [index])

// Automatické posouvání
useEffect( () => {
  let setIntervalID = setInterval( () => {
    setIndex(index + 1)
  }, 3000)
  return () => clearInterval(setIntervalID)
}, [index])
```

## 110. React - Projekt pokročilý slider - dostylování aplikace

Video: <https://youtu.be/9eGNIwDBQQY>

index.css

```
body {
  background-color: #141414;
  margin: 0;
}

.slider {
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items: center;
  height: 100vh;
}
```

OneMovieSlider.css

```
.all-movies {
  text-align: center;
  color: #e5e5e5;
  box-shadow: 0px 0px 15px #e50914;
  padding: 20px;
}

.all-movies button {
  background-color: transparent;
  border: none;
  font-size: 35px;
  color: #e50914;
  cursor: pointer;
}

.all-movies-content {
  display: flex;
  flex-direction: row;
  /* border: 2px solid red; */
  width: 500px;
  overflow: hidden;
  height: 400px;
  text-align: center;
  position: relative;
```

```
}  
  
.all-movies-content article {  
  position: absolute;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  opacity: 0;  
  transition: all 0.5s linear;  
}  
  
.all-movies-content img {  
  width: 80%;  
  height: 225px;
```

## **111. React - Projekt pokročilý slider - shrnutí**

Video: <https://youtu.be/YYQ0r0BDE6w>

## **112. React - Závěrečné video ke kurzu React pro začátečníky**

Video: <https://youtu.be/G-ect-HWaGk>